

design-detail

- B.PREDICT 詳細設計書
 - 1. データベース定義
 - 1.1 マスタ（恒久）
 - 1.2 マスタ（年度断面・履歴）
 - 1.3 ファクト
 - 1.4 派生
 - 1.5 予測
 - 1.6 評価
 - 1.7 運用
 - 1.8 トリガ
 - 2. 特徴量定義
 - 2.1 共通規約
 - 2.2 チーム勝敗モデルの特徴量
 - 2.3 個人モデルの特徴量
 - 2.4 個人スタッツの整合化
 - 2.5 Elo の算出
 - 2.6 当季集計の縮約
 - 2.7 根拠のグループ集約
 - 3. API 仕様
 - 3.1 共通
 - 3.2 入力検証
 - 3.3 公開エンドポイント
 - 3.4 内部エンドポイント
 - 3.5 キャッシュとレート制限
 - 3.6 セキュリティヘッダ
 - 4. バッチ処理詳細
 - 4.1 ワークフロー定義
 - 4.2 daily_ingest
 - 4.3 スクレイピングクライアント
 - 4.4 パーサと値域検証
 - 4.5 学習ジョブ
 - 4.6 採用基準
 - 4.7 LightGBM パラメータ
 - 4.8 backfill
 - 5. 画面詳細
 - 5.1 コンポーネント構成
 - 5.2 マークアップ規約
 - 5.3 主要コンポーネント
 - 5.4 テーマ実装
 - 5.5 数値表示
 - 5.6 データ取得
 - 6. テスト設計
 - 6.1 リーク検証（最重要）
 - 6.2 freeze の検証
 - 6.3 冪等性
 - 6.4 境界値・異常系
 - 6.4.1 個人スタッツの整合性（必須）

- 6.5 回帰
- 6.6 パーサ
- 6.7 API・画面
- 6.8 テストデータ
- 7. セキュリティ詳細
 - 7.1 public リポジトリでの禁止事項
 - 7.2 認証
 - 7.3 漏洩経路の遮断
- 8. 運用手順
 - 8.1 初期構築
 - 8.2 ローカル開発
 - 8.3 障害対応
 - 8.4 モデルのロールバック
- 9. 実装順序

B.PREDICT 詳細設計書

項目	内容
版数	1.3
作成日	2026-09-19
改訂	v1.1: 9領域レビューの指摘を反映（DDL全面改訂） / v1.2: 個人スタッツをフルボックススコアに拡張 / v1.3: 実装前検証の結果を反映（整合化アルゴリズム、DDLの試投数+成功率化、子テーブル凍結、バッチサイズ、WAF、Next.js 16、実装順序）
上位文書	docs/design-basic.md

1. データベース定義

D1 (SQLite) を使用する。日時は ISO 8601 文字列 (UTC)、日付は YYYY-MM-DD。

ID 体系の方針: マスタ・試合の `id` は公式サイトの ID を採用し、独自採番しない。公式 ID が存在しないエンティティ（会場など）のみ `v_` プレフィックスで採番し、`venue_source_keys` で名寄せする。

列挙値には必ず **CHECK 制約** を付ける。スクレイピングは入力信用できないパイプラインであり、値域の防壁をパーサだけに置くのは弱い。

1.1 マスタ（恒久）

```

-- 恒久的なクラブ。名称変更・リーグ移動があっても不変
CREATE TABLE clubs (
  id          TEXT PRIMARY KEY,
  slug        TEXT NOT NULL UNIQUE,
  name        TEXT NOT NULL,                -- 現在の表示名
  created_at  TEXT NOT NULL DEFAULT (datetime('now')),
  updated_at  TEXT NOT NULL DEFAULT (datetime('now'))
);

-- 恒久的な選手。人物マスタに純化する
CREATE TABLE players (
  id          TEXT PRIMARY KEY,            -- 公式サイト of 選手ID
  name        TEXT NOT NULL,
  height_cm   INTEGER,
  created_at  TEXT NOT NULL DEFAULT (datetime('now')),
  updated_at  TEXT NOT NULL DEFAULT (datetime('now'))
);

CREATE TABLE venues (
  id          TEXT PRIMARY KEY,
  name        TEXT NOT NULL,              -- 現在の表示名
  prefecture  TEXT,
  lat         REAL,
  lng         REAL,
  created_at  TEXT NOT NULL DEFAULT (datetime('now')),
  updated_at  TEXT NOT NULL DEFAULT (datetime('now'))
);

CREATE TABLE seasons (
  id          TEXT PRIMARY KEY,            -- '2026-27-PREMIER'
  label       TEXT NOT NULL,              -- '2026-27'
  league      TEXT NOT NULL
    CHECK (league IN ('B1', 'B2', 'B3', 'PREMIER', 'ONE', 'NEXT')),
  start_date  TEXT NOT NULL,
  end_date    TEXT NOT NULL
);

```

`seasons.id` にリーグを含める。旧版の `'2026-27'` 単独では、同一シーズンの PREMIER と ONE を同時に持てず、B.ONE 拡張時に主キー変更＝全ファクトテーブルの FK 移行が必要になる。実データ投入前の今なら修正コストはゼロ。

1.2 マスタ（年度断面・履歴）

```

-- シーズンごとのクラブ断面。名称・リーグ・本拠は年度で変わる
CREATE TABLE club_seasons (
  club_id          TEXT NOT NULL REFERENCES clubs(id),
  season_id        TEXT NOT NULL REFERENCES seasons(id),
  name             TEXT NOT NULL,
  short_name       TEXT NOT NULL,
  league           TEXT NOT NULL
                    CHECK (league IN ('B1','B2','B3','PREMIER','ONE','NEXT')),
  primary_venue_id TEXT REFERENCES venues(id),
  color_primary     TEXT,
  color_secondary   TEXT,
  PRIMARY KEY (club_id, season_id)
);

CREATE INDEX idx_club_seasons_league ON club_seasons(season_id, league);

-- 公式サイトのチームID (旧B1 / 新リーグ) を club_id に解決する
CREATE TABLE club_source_ids (
  source_id TEXT PRIMARY KEY,
  club_id   TEXT NOT NULL REFERENCES clubs(id),
  valid_from TEXT NOT NULL,
  valid_to   TEXT NOT NULL DEFAULT '9999-12-31',
  note      TEXT
);

CREATE INDEX idx_csi_club ON club_source_ids(club_id);

-- 選手の所属断面。シーズン途中の移籍にも対応する
CREATE TABLE player_seasons (
  player_id TEXT NOT NULL REFERENCES players(id),
  season_id TEXT NOT NULL REFERENCES seasons(id),
  club_id   TEXT NOT NULL REFERENCES clubs(id),
  number    TEXT,
  position  TEXT CHECK (position IN ('PG','SG','SF','PF','C')),
  roster_type TEXT CHECK (roster_type IN ('JP','NATURALIZED','ASIA','FOREIGN')),
  joined_on TEXT,
  left_on   TEXT,
  PRIMARY KEY (player_id, season_id, club_id)
);

CREATE INDEX idx_pseason_club ON player_seasons(club_id, season_id);

-- 会場の改称・収容人数変更の履歴。過去試合は当時の値で表示する
CREATE TABLE venue_revisions (
  venue_id TEXT NOT NULL REFERENCES venues(id),
  valid_from TEXT NOT NULL,
  valid_to   TEXT NOT NULL DEFAULT '9999-12-31',
  name       TEXT NOT NULL,
  capacity   INTEGER,
  PRIMARY KEY (venue_id, valid_from)
);

-- 会場名の表記ゆれを名寄せする
CREATE TABLE venue_source_keys (
  source_name TEXT PRIMARY KEY,
  venue_id    TEXT NOT NULL REFERENCES venues(id)
);

```

取り込み時に `club_source_ids` で `club_id` に解決してから保存する。旧IDのまま保存すると `team_ratings.team_id` が分断され、旧B1の Elo を新リーグへ引き継げない。

日本のアリーナは命名権で頻繁に改称する。`venues.name` を上書きすると過去試合の会場表示が遡って変わり、収容人数の変更で過去の動員率も歪む。

1.3 ファクト

```
CREATE TABLE games (  
  id TEXT PRIMARY KEY, -- 公式サイトの試合ID  
  season_id TEXT NOT NULL REFERENCES seasons(id),  
  league TEXT NOT NULL, -- API応答と一致させるため非正規化  
  game_date TEXT NOT NULL, -- 変更されうる属性  
  tipoff_at TEXT NOT NULL,  
  finished_at TEXT, -- 実績。なければ tipoff_at + 2h を推定値とする  
  home_club_id TEXT NOT NULL REFERENCES clubs(id),  
  away_club_id TEXT NOT NULL REFERENCES clubs(id),  
  venue_id TEXT REFERENCES venues(id),  
  is_primary_venue INTEGER NOT NULL DEFAULT 1 CHECK (is_primary_venue IN (0,1)),  
  series_game_no INTEGER,  
  status TEXT NOT NULL  
    CHECK (status IN ('SCHEDULED', 'FINISHED', 'POSTPONED', 'CANCELLED')),  
  rescheduled_to TEXT REFERENCES games(id), -- 延期先。旧行は POSTPONED で残す  
  home_score INTEGER,  
  away_score INTEGER,  
  attendance INTEGER,  
  spectator_restricted INTEGER NOT NULL DEFAULT 0  
    CHECK (spectator_restricted IN (0,1)),  
  result_revision INTEGER NOT NULL DEFAULT 0, -- スコア訂正のたびに +1  
  source_url TEXT,  
  fetched_at TEXT,  
  created_at TEXT NOT NULL DEFAULT (datetime('now')),  
  updated_at TEXT NOT NULL DEFAULT (datetime('now')),  
  CHECK (home_club_id <> away_club_id),  
  UNIQUE (season_id, game_date, home_club_id, away_club_id) -- 重複検出用。upsertキーではない  
);  
  
CREATE INDEX idx_games_date ON games(game_date, status);  
CREATE INDEX idx_games_finished ON games(finished_at);  
CREATE INDEX idx_games_home ON games(home_club_id, game_date);  
CREATE INDEX idx_games_away ON games(away_club_id, game_date);
```

upsert キーは **id** (公式試合ID) とする。自然キーに **game_date** を含めると、延期で日付が変わった瞬間に別レコードとして挿入され、旧行が **SCHEDULED** のまま永久に残る。ゴースト試合の予測が的中率の分母を汚染する。

公式試合IDが取得できない場合に限り自然キーを使うが、その場合は日程変更の名寄せ処理を明示的に実装する。

```
-- チーム視点の試合行。OR 条件を消し、未実施試合も扱える  
CREATE TABLE team_games (  
  game_id TEXT NOT NULL REFERENCES games(id),  
  club_id TEXT NOT NULL REFERENCES clubs(id),  
  opponent_id TEXT NOT NULL REFERENCES clubs(id),  
  season_id TEXT NOT NULL REFERENCES seasons(id),  
  game_date TEXT NOT NULL,  
  finished_at TEXT,  
  is_home INTEGER NOT NULL CHECK (is_home IN (0,1)),  
  result INTEGER CHECK (result IN (0,1)), -- NULL = 未実施  
  margin INTEGER,  
  PRIMARY KEY (club_id, game_date, game_id)  
);  
  
CREATE INDEX idx_team_games_finished ON team_games(club_id, finished_at);
```

```

CREATE TABLE team_game_stats (
  game_id    TEXT NOT NULL REFERENCES games(id),
  club_id    TEXT NOT NULL REFERENCES clubs(id),
  game_date  TEXT NOT NULL,                -- JOIN とソートを消すための非正規化
  is_home    INTEGER NOT NULL CHECK (is_home IN (0,1)),
  pts        INTEGER,
  fg2m       INTEGER, fg2a       INTEGER,   -- 選手側と同じ粒度で持つ (整合化の基準になる)
  fg3m       INTEGER, fg3a       INTEGER,
  ftm        INTEGER, fta        INTEGER,
  oreb       INTEGER, dreb       INTEGER,
  ast        INTEGER, tov        INTEGER, stl INTEGER, blk INTEGER,
  pf         INTEGER, fd         INTEGER,
  possessions REAL,
  fetched_at TEXT NOT NULL,
  updated_at TEXT NOT NULL DEFAULT (datetime('now')),
  PRIMARY KEY (game_id, club_id),
  CHECK (fg2m IS NULL OR fg2a IS NULL OR fg2m <= fg2a),
  CHECK (fg3m IS NULL OR fg3a IS NULL OR fg3m <= fg3a),
  CHECK (ftm IS NULL OR fta IS NULL OR ftm <= fta),
  CHECK (pts IS NULL OR (pts >= 0 AND pts <= 250))
);

CREATE INDEX idx_tgs_club ON team_game_stats(club_id, game_date);

CREATE TABLE player_game_stats (
  game_id    TEXT NOT NULL REFERENCES games(id),
  player_id  TEXT NOT NULL REFERENCES players(id),
  club_id    TEXT NOT NULL REFERENCES clubs(id),
  game_date  TEXT NOT NULL,
  started    INTEGER CHECK (started IN (0,1)),
  minutes    REAL CHECK (minutes IS NULL OR (minutes >= 0 AND minutes <= 60)),
  -- シュート。2P と 3P を別建てで持ち、FG は導出する
  fg2m       INTEGER, fg2a       INTEGER,
  fg3m       INTEGER, fg3a       INTEGER,
  ftm        INTEGER, fta        INTEGER,
  -- リバウンド
  oreb       INTEGER, dreb       INTEGER,
  -- プレー
  ast        INTEGER, tov        INTEGER, stl INTEGER, blk INTEGER,
  -- ファウル
  pf         INTEGER,            -- F : 自分が犯したファウル
  fd         INTEGER,            -- FD : 被ファウル数
  -- 実績としてのみ保持 (予測しない)
  plus_minus INTEGER,
  pts        INTEGER,           -- 取得値。恒等式の検証に使う
  fetched_at TEXT NOT NULL,
  updated_at TEXT NOT NULL DEFAULT (datetime('now')),
  PRIMARY KEY (game_id, player_id),
  CHECK (fg2m IS NULL OR fg2a IS NULL OR fg2m <= fg2a),
  CHECK (fg3m IS NULL OR fg3a IS NULL OR fg3m <= fg3a),
  CHECK (ftm IS NULL OR fta IS NULL OR ftm <= fta),
  CHECK (pf IS NULL OR (pf >= 0 AND pf <= 6))
);

CREATE INDEX idx_pgs_player ON player_game_stats(player_id, game_date);

```

fgm / fga / reb の列は持たない。fg2m + fg3m / fg2a + fg3a / oreb + dreb で導出できる冗長列であり、持つと不整合の余地が生まれる。

取り込み時に以下の恒等式を検証する。不一致は当該試合をスキップする。

```
pts = fg2m × 2 + fg3m × 3 + ftm
fg2m ≤ fg2a, fg3m ≤ fg3a, ftm ≤ fta
チーム集計の各項目 = その試合の所属選手の合計
```

FD（被ファウル数）と 2FGM/2FGA の取得可否は Phase 0（P0-9）で検証する。 用語集には定義があるが、個別試合のボックススコアページでの公開状況は未確認である。取得できない場合は該当項目を予測対象から外す。

`possessions` は取り込み時に算出する。必要項目が欠ける場合は NULL とし、依存する特徴量を無効化する。

```
fga = fg2a + fg3a
possessions = fga - oreb + tov + 0.44 * fta
```

係数 0.44 は NBA のフリースロー試投係数であり、日本のバスケットに当てはまる保証はない。**Phase 0（P0-7）で実データから再推定する。**

```
CREATE TABLE game_entries (
  game_id TEXT NOT NULL REFERENCES games(id),
  player_id TEXT NOT NULL REFERENCES players(id),
  status TEXT NOT NULL CHECK (status IN ('ENTRY', 'OUT', 'UNKNOWN')),
  source TEXT NOT NULL CHECK (source IN ('OFFICIAL', 'ESTIMATED')),
  confidence REAL CHECK (confidence IS NULL OR (confidence BETWEEN 0 AND 1)),
  fetched_at TEXT NOT NULL,
  PRIMARY KEY (game_id, player_id)
);
```

取得ごとに当該試合の全行を DELETE してから INSERT する（洗い替え）。 公式エントリーはエントリーした選手だけを返すため、追加のみの upsert では「出場するはず」と推定した行のうち実際には登録外だった選手の行が残り続け、`is_provisional` が永久に解除されない。

`source = 'ESTIMATED'` の行が1件でも残る試合は `is_provisional = 1` とする。

1.4 派生

```
CREATE TABLE team_ratings (
  club_id TEXT NOT NULL REFERENCES clubs(id),
  as_of_date TEXT NOT NULL,
  season_id TEXT NOT NULL REFERENCES seasons(id),
  elo REAL NOT NULL,
  off_rating REAL,
  def_rating REAL,
  pace REAL,
  games_played INTEGER NOT NULL,
  PRIMARY KEY (club_id, as_of_date)
);
```

試合日ごとのスナップショット。特徴量生成時は `as_of_date < 対象試合日` の最新行を参照する。この構造自体がリーク防止になっている。**Elo をその場で計算する実装にしない**（全試合を走査する過程で対象試合を含めてしまう事故が起きる）。

1.5 予測

```

CREATE TABLE predictions (
  id TEXT PRIMARY KEY,
  game_id TEXT NOT NULL REFERENCES games(id),
  season_id TEXT NOT NULL,          -- accuracy 集計用に非正規化
  model_version TEXT NOT NULL REFERENCES model_versions(version),
  revision INTEGER NOT NULL,        -- 世代通番 1,2,3...
  run_id TEXT NOT NULL,             -- ingestion_logs.id
  predicted_at TEXT NOT NULL,
  as_of TEXT NOT NULL,              -- 特徴量が参照してよい上限時刻
  data_as_of TEXT NOT NULL,         -- 実行時点でDBにあった最新試合の終了時刻
  home_win_prob REAL NOT NULL CHECK (home_win_prob BETWEEN 0 AND 1),
  pred_margin REAL,
  pred_total REAL,
  pred_home_score REAL,
  pred_away_score REAL,
  is_provisional INTEGER NOT NULL DEFAULT 1 CHECK (is_provisional IN (0,1)),
  is_final INTEGER NOT NULL DEFAULT 0 CHECK (is_final IN (0,1)),
  is_active INTEGER NOT NULL DEFAULT 1 CHECK (is_active IN (0,1)),
  feature_snapshot TEXT NOT NULL,    -- JSON
  created_at TEXT NOT NULL DEFAULT (datetime('now')),
  UNIQUE (game_id, model_version, revision)
);

-- 1試合につき有効な予測は常に1本
CREATE UNIQUE INDEX uq_pred_active ON predictions(game_id) WHERE is_active = 1;
-- 確定済みも1本
CREATE UNIQUE INDEX uq_pred_final ON predictions(game_id) WHERE is_final = 1;
CREATE INDEX idx_pred_eval ON predictions(model_version, is_final, season_id);

```

away_win_prob 列は持たない (1 - home_win_prob で導出できる冗長列)。

home_win_prob がどちらの経路 (Winner 直接 / Margin から導出) で算出されたかは model_versions.win_prob_source に記録する。予測行ごとには持たない (同一モデルバージョン内では常に同一のため)。

```

-- 整合化の目標値。チーム側も「試投数 + 成功率」で持つ
CREATE TABLE prediction_team_targets (
  prediction_id TEXT NOT NULL REFERENCES predictions(id),
  club_id TEXT NOT NULL REFERENCES clubs(id),
  is_home INTEGER NOT NULL CHECK (is_home IN (0,1)),

  tgt_fg2a REAL NOT NULL CHECK (tgt_fg2a >= 0),
  tgt_fg3a REAL NOT NULL CHECK (tgt_fg3a >= 0),
  tgt_fta REAL NOT NULL CHECK (tgt_fta >= 0),
  tgt_fg2_pct REAL NOT NULL CHECK (tgt_fg2_pct BETWEEN 0 AND 1),
  tgt_fg3_pct REAL NOT NULL CHECK (tgt_fg3_pct BETWEEN 0 AND 1),
  tgt_ft_pct REAL NOT NULL CHECK (tgt_ft_pct BETWEEN 0 AND 1),

  tgt_oreb REAL NOT NULL CHECK (tgt_oreb >= 0),
  tgt_dreb REAL NOT NULL CHECK (tgt_dreb >= 0),
  tgt_ast REAL NOT NULL CHECK (tgt_ast >= 0),
  tgt_tov REAL NOT NULL CHECK (tgt_tov >= 0),
  tgt_stl REAL NOT NULL CHECK (tgt_stl >= 0),
  tgt_blk REAL NOT NULL CHECK (tgt_blk >= 0),
  tgt_pf REAL NOT NULL CHECK (tgt_pf >= 0),
  tgt_fd REAL NOT NULL CHECK (tgt_fd >= 0),

  PRIMARY KEY (prediction_id, club_id)
);

```

目標値を「成功数」ではなく「試投数 + 成功率」で持つ理由。成功数と試投数を独立に保存すると、約2%の確率で $\text{成功数} > \text{試投数}$ という実行不能目標が生まれる。この状態ではどんな整合化アルゴリズムも目標に到達できない。試投数（非負）と成功率（0～1）の組で持てば、 $\text{成功数} = \text{率} \times \text{試投数} \leq \text{試投数}$ が構造的に保証される。CHECK 制約はこの構造を DB レベルで固定するものであり、外さない。

得点の目標値は別列として持たず、 $\text{tgt_fg2_pct} \times \text{tgt_fg2a} \times 2 + \text{tgt_fg3_pct} \times \text{tgt_fg3a} \times 3 + \text{tgt_ft_pct} \times \text{tgt_fta}$ で導出する。この導出値が $\text{predictions.pred_home_score} / \text{pred_away_score}$ と ± 0.5 点以内で一致することを `test_team_targets_are_feasible` (A-16) で検証する。

運用規則

- 再推論時は旧行の `is_active` を 0 にし、新行を `revision + 1` で追加する。この2文は必ず単一の D1 `batch()` に入れる（D1 にはリクエストを跨ぐトランザクションがない）
- 試合開始時刻を過ぎたら、その時点の `is_active = 1` の行に `is_final = 1` を立てる
- `is_final = 1` の行は一切更新・削除しない。トリガで禁止し、アプリ層の規律だけに頼らない
- 的中率の算出は `is_final = 1` の行のみを使用する

```
CREATE TABLE player_predictions (  
  id TEXT PRIMARY KEY,  
  prediction_id TEXT NOT NULL REFERENCES predictions(id), -- 親の世代に紐付ける  
  game_id TEXT NOT NULL REFERENCES games(id),  
  player_id TEXT NOT NULL REFERENCES players(id),  
  club_id TEXT NOT NULL REFERENCES clubs(id),  
  model_version TEXT NOT NULL,  
  revision INTEGER NOT NULL,  
  predicted_at TEXT NOT NULL,  
  
  avail_prob REAL NOT NULL CHECK (avail_prob BETWEEN 0 AND 1),  
  pred_minutes REAL NOT NULL CHECK (pred_minutes >= 0), -- 整合化後の期待出場時間  
  
  -- 整合化後の試投数（カウント。期待値なので REAL）  
  pred_fg2a REAL NOT NULL CHECK (pred_fg2a >= 0),  
  pred_fg3a REAL NOT NULL CHECK (pred_fg3a >= 0),  
  pred_fta REAL NOT NULL CHECK (pred_fta >= 0),  
  
  -- 整合化後の成功率（0～1）。成功数はこれと試投数の積で導出する  
  pred_fg2_pct REAL NOT NULL CHECK (pred_fg2_pct BETWEEN 0 AND 1),  
  pred_fg3_pct REAL NOT NULL CHECK (pred_fg3_pct BETWEEN 0 AND 1),  
  pred_ft_pct REAL NOT NULL CHECK (pred_ft_pct BETWEEN 0 AND 1),  
  
  -- その他のカウント  
  pred_oreb REAL NOT NULL CHECK (pred_oreb >= 0),  
  pred_dreb REAL NOT NULL CHECK (pred_dreb >= 0),  
  pred_ast REAL NOT NULL CHECK (pred_ast >= 0),  
  pred_tov REAL NOT NULL CHECK (pred_tov >= 0),  
  pred_stl REAL NOT NULL CHECK (pred_stl >= 0),  
  pred_blk REAL NOT NULL CHECK (pred_blk >= 0),  
  pred_pf REAL NOT NULL CHECK (pred_pf >= 0),  
  pred_fd REAL NOT NULL CHECK (pred_fd >= 0),  
  
  -- 誤差の目安（当該選手の直近N試合の絶対誤差の中央値）。主要4項目のみ  
  err_minutes REAL, err_pts REAL, err_reb REAL, err_ast REAL,  
  
  is_provisional INTEGER NOT NULL DEFAULT 1,  
  is_final INTEGER NOT NULL DEFAULT 0,  
  is_active INTEGER NOT NULL DEFAULT 1,  
  UNIQUE (game_id, player_id, model_version, revision)  
);  
  
CREATE UNIQUE INDEX uq_ppred_active ON player_predictions(game_id, player_id) WHERE is_active = 1;
```

成功数（`pred_fg2m` 等）の列を持たない。旧版は成功数と試投数の両方を列に持ち、`pred_fg2m <= pred_fg2a` を CHECK で守っていた。この構造では

整合化の各ステップで両者を別々にスケールする必要があり、**1.0 に張り付いた成功率を持つ選手が出た時点で合計を目標に合わせられなくなる**（実測で失敗ケースの73%がFTで発生）。試投数と成功率の組で持てば、成功率をロジット空間でシフトするだけで制約が自動的に満たされ、CHECKによる事後防御そのものが不要になる。

`pred_pts / pred_fgm / pred_fga / pred_reb` の列も持たない。すべて下表のとおり導出できる。列として持つと、整合化の後に恒等式が崩れても検出できなくなる。

導出項目	式
2FGM / 3FGM / FTM	<code>pred_fg2_pct × pred_fg2a / pred_fg3_pct × pred_fg3a / pred_ft_pct × pred_fta</code>
PTS	<code>2FGM × 2 + 3FGM × 3 + FTM</code>
FGM / FGA	<code>2FGM + 3FGM / pred_fg2a + pred_fg3a</code>
TR	<code>pred_oreb + pred_dreb</code>
FG%（表示用）	<code>FGM / FGA</code>
EFG%	<code>(FGM + 0.5 × 3FGM) / FGA</code>
TS%	<code>PTS / (2 × (FGA + 0.44 × pred_fta))</code>

保存する成功率を画面にそのまま出さない。 これは「この選手のこの試合における期待成功率」という連続量であり、表示する `FGM / FGA` とは別物である。画面では必ず分数と併記し、試投数が閾値未満なら率を出さない（5.3 PlayerStatTable）。

+/- の予測列を持たない。 1試合の個人プラスマイナスは分散が極端に大きく（MAE 9～12点規模）、「チーム得点差予測 × 出場時間シェア」以上の情報を持たない。実績（`player_game_stats.plus_minus`）のみ表示する。

行数の見積もり: 1試合あたり最大24人、日次で7日分×最大13試合 = 約2,200行/日。D1 の書込枠（10万行/日）に対して問題ない。

```
CREATE TABLE prediction_reasons (  
  prediction_id TEXT NOT NULL REFERENCES predictions(id),  
  rank          INTEGER NOT NULL,  
  group_key     TEXT NOT NULL,      -- 'TEAM_STRENGTH' | 'SCHEDULE' | 'PLAYER' | 'VENUE'  
  label_ja      TEXT NOT NULL,  
  value_text    TEXT NOT NULL,  
  favors        TEXT NOT NULL CHECK (favors IN ('HOME', 'AWAY')),  
  contribution  REAL NOT NULL,     -- グループ集約したSHAP値（ログオッズ空間）  
  base_value    REAL NOT NULL,     -- explainer の期待値。事後検証用  
  PRIMARY KEY (prediction_id, rank)  
);
```

1.6 評価

```

CREATE TABLE model_versions (
  version      TEXT PRIMARY KEY,          -- 'winner-v1.0.0'
  model_type   TEXT NOT NULL
              CHECK (model_type IN ('WINNER', 'MARGIN', 'TOTAL', 'TEAM_RATE',
                                    'PLAYER_AVAIL', 'PLAYER_MIN', 'PLAYER_RATE')),
  target       TEXT NOT NULL DEFAULT '',  -- *_RATE のみ: 'fg2a' | 'fg2_pct' | ... の14種
  league       TEXT NOT NULL DEFAULT 'PREMIER',
  win_prob_source TEXT CHECK (win_prob_source IN ('WINNER', 'MARGIN')),
                                      -- WINNER/MARGIN のみ。採用した勝率導出経路
  margin_sigma REAL,                    -- MARGIN のみ。P(home)= $\Phi$ (margin/ $\sigma$ ) の  $\sigma$ 
  algo         TEXT NOT NULL,
  trained_at   TEXT NOT NULL,
  train_rows   INTEGER NOT NULL,
  train_range  TEXT NOT NULL,
  eval_window  TEXT NOT NULL,          -- 比較の公平性のため固定した評価対象
  params       TEXT NOT NULL,          -- JSON。seed 系を必ず含める
  feature_list TEXT NOT NULL,          -- JSON配列
  feature_null_rates TEXT,             -- JSON。欠損率30%超の検出用
  cv_accuracy  REAL,
  cv_brier     REAL,
  cv_logloss   REAL,
  cv_ece       REAL,                   -- 等頻度10ビン
  baseline_home_accuracy REAL,
  baseline_elo_brier REAL,             -- Elo単体ロジスティック回帰
  artifact_text TEXT,                  -- LightGBM の save_model() 出力
  artifact_sha256 TEXT,
  calibrator   TEXT,                   -- 校正器のパラメータ。使う場合のみ
  is_active    INTEGER NOT NULL DEFAULT 0 CHECK (is_active IN (0,1)),
  notes       TEXT
);
-- 同一 (model_type, target, league) で有効モデルは常に1本
CREATE UNIQUE INDEX uq_model_active
  ON model_versions(model_type, target, league) WHERE is_active = 1;

```

モデルバイナリは D1 に格納する。GitHub Actions の artifact はデフォルト90日で失効し、別ワークフローから取得するには run-id 解決が必要なため、日次推論の依存先として成立しない。LightGBM の `save_model()` はテキスト形式で数百KB～数MBに収まり、5GBのストレージ枠に対して影響は無視できる。

ただし D1 の1行上限は 2,000,000 バイトである。書式からの推定では `num_leaves=7` / 木300～800本で 0.27～0.70MB に収まるが、木が2,000本まで伸びると 1.75MB (上限の88%)、`num_leaves=15` / 1,500本では 2.56MB で上限を超える。

```

-- 登録時のサイズガード (アプリ層でも同じ判定を行い、二重化する)
CREATE TRIGGER trg_model_artifact_size
BEFORE INSERT ON model_versions
WHEN NEW.artifact_text IS NOT NULL AND length(NEW.artifact_text) > 1572864 -- 1.5 MiB
BEGIN
  SELECT RAISE(ABORT, 'artifact_text exceeds 1.5MB limit');
END;

```

上限を超えた場合は登録を拒否し、現行モデルを継続使用する (学習ジョブは `PARTIAL` + exit 1 で終える)。`num_boost_round` の上限は 1,200 とし、`num_leaves` を 7 から増やす変更は artifact サイズの再測定とセットでのみ許可する。実サイズは Phase 0 (P0-12) で測定する。

`uq_model_active` があれば、ロールバック時に「先に 0 にしてから 1 にする」順序でのみ成功し、有効モデル2本という状態が構造的に作れなくなる。

```
-- 推論に使ったモデル一式の記録。予測1本は複数モデルの合成である
CREATE TABLE prediction_model_bundle (
  prediction_id TEXT NOT NULL REFERENCES predictions(id),
  model_type    TEXT NOT NULL,
  target        TEXT NOT NULL DEFAULT '',
  model_version TEXT NOT NULL REFERENCES model_versions(version),
  PRIMARY KEY (prediction_id, model_type, target)
);
```

predictions.model_version だけでは出自を復元できない。1本の予測は Winner / Margin / Total / TeamRate×14 / PlayerAvail / PlayerMin / PlayerRate×14 の合成であり、単一の列にはチームモデルのバージョンしか書けない。個人スタッツのどのモデルで出た値かが記録されないと、「あの試合の個人予測はどのモデルか」を後から特定できず、モデル別の精度比較（/accuracy の byModel）も個人スタッツについては成立しない。

predictions.model_version は「代表バージョン（Winner または Margin のうち採用経路のもの）」として残し、全体は prediction_model_bundle で記録する。1予測あたり最大32行増えるが、日次で最大13試合 × 32 = 約416行であり書込枠に対して問題ない。

```
CREATE TABLE prediction_results (
  prediction_id    TEXT PRIMARY KEY REFERENCES predictions(id),
  game_id          TEXT NOT NULL REFERENCES games(id),
  season_id        TEXT NOT NULL,
  model_version     TEXT NOT NULL,
  home_win_prob    REAL NOT NULL,
  prob_bucket      INTEGER NOT NULL,
  outcome           TEXT NOT NULL
  CHECK (outcome IN ('WIN', 'LOSS', 'VOID')),
  predicted_home_win INTEGER,
  actual_home_win   INTEGER,
  is_correct         INTEGER,
  brier              REAL,
  score_mae          REAL,
  was_provisional    INTEGER NOT NULL,
  evaluated_at       TEXT NOT NULL DEFAULT (datetime('now'))
);
CREATE INDEX idx_predres_season ON prediction_results(model_version, season_id);
CREATE INDEX idx_predres_calib  ON prediction_results(model_version, prob_bucket);
```

中止・延期試合は outcome = 'VOID' とし、的中率の母数から除外する。旧版は actual_home_win INTEGER NOT NULL だったため、中止試合が1件出ると NOT NULL 制約違反でバッチが毎日失敗し続け、取り込みが完全停止していた。

```
-- 日次で洗い替える集計層
CREATE TABLE accuracy_summary (
  scope      TEXT NOT NULL, -- 'OVERALL' | 'SEASON' | 'MODEL' | 'BUCKET' | 'PROVISIONAL'
  scope_key   TEXT NOT NULL,
  model_version TEXT,
  n           INTEGER NOT NULL,
  accuracy    REAL NOT NULL,
  brier       REAL NOT NULL,
  actual_rate REAL,
  baseline_accuracy REAL,
  updated_at  TEXT NOT NULL DEFAULT (datetime('now')),
  PRIMARY KEY (scope, scope_key, model_version)
);
```

1.7 運用

```

CREATE TABLE ingestion_logs (
  id          TEXT PRIMARY KEY,
  job         TEXT NOT NULL,
  started_at  TEXT NOT NULL,
  finished_at TEXT,
  status      TEXT NOT NULL
  CHECK (status IN ('RUNNING', 'SUCCESS', 'PARTIAL', 'FAILED', 'ABORTED')),
  rows_affected INTEGER,
  d1_rows_read INTEGER,          -- 無料枠の監視用
  error_type  TEXT,              -- 例外の型名のみ
  error_message TEXT            -- 自前の短いメッセージのみ
);
CREATE INDEX idx_inglog_job ON ingestion_logs(job, started_at DESC);

```

例外オブジェクトをそのまま保存しない。D1 REST API の URL にはアカウントIDとDB IDが含まれるため、`str(e)` を保存して `/health` で公開すると、それらが公開APIから漏れる。型名と自前メッセージに限定し、URLを残す場合はホスト名とパスのみでクエリ文字列とID部分をマスクする。

1.8 トリガ

```

CREATE TRIGGER trg_predictions_final_immutable
BEFORE UPDATE ON predictions
WHEN OLD.is_final = 1
BEGIN
  SELECT RAISE(ABORT, 'final prediction is immutable');
END;

CREATE TRIGGER trg_predictions_final_nodelete
BEFORE DELETE ON predictions
WHEN OLD.is_final = 1
BEGIN
  SELECT RAISE(ABORT, 'final prediction cannot be deleted');
END;

-- 子テーブルにも例外なく同じ凍結を適用する
CREATE TRIGGER trg_ppred_final_immutable
BEFORE UPDATE ON player_predictions
WHEN OLD.is_final = 1
BEGIN
  SELECT RAISE(ABORT, 'final player prediction is immutable');
END;

CREATE TRIGGER trg_ppred_final_nodelete
BEFORE DELETE ON player_predictions
WHEN OLD.is_final = 1
BEGIN
  SELECT RAISE(ABORT, 'final player prediction cannot be deleted');
END;

-- prediction_reasons は is_final を持たないため、親を参照して判定する
CREATE TRIGGER trg_reasons_final_immutable
BEFORE UPDATE ON prediction_reasons
WHEN (SELECT is_final FROM predictions WHERE id = OLD.prediction_id) = 1
BEGIN
  SELECT RAISE(ABORT, 'reasons of a final prediction are immutable');
END;

CREATE TRIGGER trg_reasons_final_nodelete
BEFORE DELETE ON prediction_reasons

```

```

WHEN (SELECT is_final FROM predictions WHERE id = OLD.prediction_id) = 1
BEGIN
    SELECT RAISE(ABORT, 'reasons of a final prediction cannot be deleted');
END;

CREATE TRIGGER trg_team_targets_final_immutable
BEFORE UPDATE ON prediction_team_targets
WHEN (SELECT is_final FROM predictions WHERE id = OLD.prediction_id) = 1
BEGIN
    SELECT RAISE(ABORT, 'team targets of a final prediction are immutable');
END;

CREATE TRIGGER trg_team_targets_final_nodelete
BEFORE DELETE ON prediction_team_targets
WHEN (SELECT is_final FROM predictions WHERE id = OLD.prediction_id) = 1
BEGIN
    SELECT RAISE(ABORT, 'team targets of a final prediction cannot be deleted');
END;

CREATE TRIGGER trg_bundle_final_immutable
BEFORE UPDATE ON prediction_model_bundle
WHEN (SELECT is_final FROM predictions WHERE id = OLD.prediction_id) = 1
BEGIN
    SELECT RAISE(ABORT, 'model bundle of a final prediction is immutable');
END;

CREATE TRIGGER trg_bundle_final_nodelete
BEFORE DELETE ON prediction_model_bundle
WHEN (SELECT is_final FROM predictions WHERE id = OLD.prediction_id) = 1
BEGIN
    SELECT RAISE(ABORT, 'model bundle of a final prediction cannot be deleted');
END;

```

凍結に例外を設けない。旧版は `predictions` にのみトリガを置いていたが、それでは「勝率は変えられないが、個人スタッツ・根拠・目標値・モデル構成は後から書き換えられる」状態になる。根拠は「なぜその予測になったか」の記録であり、試合後に差し替えられるなら予測の不変性という主張そのものが成立しない。

表示上の誤り（ラベルの誤字など）が見つかった場合も、**過去の行は修正しない**。以後の生成ロジックのみを直す。`is_final = 1` を立てる freeze 操作自体は `predictions` の UPDATE であり、この2つのトリガは `OLD.is_final = 1` を条件にしているため、`0 → 1` の遷移は通る（`1 → 1` は通らない）。

2. 特徴量定義

2.1 共通規約

```

def build_features(game_id: str, as_of: datetime, ds: Dataset) -> dict[str, float]:
    """as_of 時点で確定している情報のみから特徴量を生成する。

    as_of  : 参照してよい情報の上限時刻 (= games.tipoff_at)
    ds      : batch/snapshot/*.parquet から読み込んだメモリ内データセット
              (D1 は読まない。CLAUDE.md 絶対ルール3)

    絞り込みは finished_at で行う。tipoff_at で絞ると、17:05開始・19:05終了の
    試合が 19:05開始の試合の確定情報として混入する。
    """

```

すべての特徴量関数は以下を守る。

- 1. 引数 `as_of` を必ず受け取る
- 2. 参照条件に `finished_at <= as_of` を含める（`tipoff_at` ではない）
- 3. 対象試合自身の `team_game_stats` / `player_game_stats` / `games.attendance` を参照しない
- 4. `status = 'SCHEDULED'` の試合を集計に含めない
- 5. 欠損時は `None` を返し、呼び出し側で既定値へ変換する（関数内で0埋めしない）
- 6. チーム所属の判定は `players` の現在の所属ではなく、`player_game_stats.club_id`（実績）または `player_seasons`（当該シーズン断面）を使う

規約6を守らないと、移籍が発生した瞬間に「そのチームの過去の主力」が誤る。移籍した選手が過去の所属チームから消え、新チームに過去の出場時間ごと移動する。`as_of` 規約には違反しないため、テストでも検出されない静かなバグになる。

D1を逐次クエリしない。 入口でテーブル単位に一括エクスポートし、以降はメモリ上で処理する。

2.2 チーム勝敗モデルの特徴量

命名規則: {カテゴリ}_{内容}_{対象}。対象は `home` / `away` / `diff`。相関を減らすため、可能な限り `diff` に集約する。

採用

キー	定義	欠損時
<code>elo_diff</code>	Elo の差	0
<code>elo_home</code> / <code>elo_away</code>	<code>team_ratings</code> の直前スナップショット	1500
<code>winrate_l5_diff</code> / <code>winrate_l10_diff</code>	直近勝率の差	0
<code>winrate_season_diff</code>	当季通算勝率の差（縮約後）	0
<code>margin_l5_diff</code> / <code>margin_season_diff</code>	平均得失点差の差	0
<code>series_game_no</code>	同一カード連戦の何戦目か	1
<code>prev_result_diff</code>	前戦の勝敗の差	0
<code>prev_margin_diff</code>	前戦の得失点差の差	0
<code>rest_days_diff</code>	休養日数の差（14日でクリップ）	0
<code>away_streak_away</code>	アウェイ側の連続アウェイ試合数	0
<code>minutes_lost_diff</code>	欠場者の直近平均出場時間の合計の差	0
<code>top_players_out_diff</code>	直近出場時間上位5名の欠場者数の差	0
<code>entry_is_official</code>	両チームのエントリーが公式確定なら1	0

`is_home` は持たない（ホーム視点固定学習のため常に1の定数列になる）。

検証

キー	定義	前提
ortg_diff / drtg_diff	100ポゼッションあたり得失点の差	possessions が算出可能
pace_home / pace_away	1試合あたりポゼッション数	同上
efg_diff tov_rate_diff oreb_rate_diff ft_rate_diff	Four Factors の差	ボックススコア取得可
sos_diff	対戦相手の平均 Elo の差	—
is_primary_venue	メイン会場なら1	会場マスタ
travel_km_diff	前戦会場からの直線距離の差	緯度経度
rank_diff	順位差	—
playoff_race_diff	プレーオフ圏との勝率差の差	—
season_progress	当季の消化率 0-1	—
day_of_week / tipoff_hour	曜日・開始時刻	—
minutes_concentration_diff	出場時間のハーフィンダール指数の差	—
foreign_count_diff / naturalized_diff	ロスター構成の差	player_seasons
venue_capacity	収容人数（当時の値）	venue_revisions
attendance_avg_home / attendance_rate_home / attendance_avg_venue	過去試合からの平均動員	観客制限期間を除外

特徴量を一度に全部入れない。 まず elo_diff と日程系と欠場系の10～15列で基礎モデルを作り、1つずつ追加して walk-forward Brier の改善を測る。8,000行に対して70列は、初期 fold（1,600行）では明確に過剰である。

2.3 個人モデルの特徴量

キー	定義	対象モデル
minutes_l5_player / minutes_l10_player	直近の平均出場時間	Avail / Minutes
games_played_ratio_l10	直近10試合の出場率	Avail
entry_status	エントリー情報（公式/推定）	Avail
days_since_last_played	最終出場からの日数	Avail
{count}_per_min_l10	カウント11項目それぞれの単位時間あたり実績（直近10試合）	Rates
{count}_per_min_season	同 当季平均	Rates
{pct}_shrunk_l10	成功率3項目のシュリンク済み実績（ (made + k×prior) / (att + k) ）	Rates
{pct}_shrunk_season	同 当季	Rates
{att}_l10	各成功率に対応する試投数（成功率モデルの学習重み）	Rates
usage_l10	使用率の近似	Rates
pred_minutes	Minutes モデルの出力	Rates
team_minutes_lost	同僚の欠場合計	Minutes / Rates
opponent_drtg / opponent_pace	相手の守備力とペース	Rates
opponent_{stat}_allowed_l10	相手が直近10試合で許した当該項目の水準	Rates
is_starter_l5	直近のスターター率	Minutes
position	ポジション（OR/BS はビッグマンに偏る）	Rates

{count} は fg2a, fg3a, fta, oreb, dreb, ast, tov, stl, blk, pf, fd の11項目。{pct} は fg2_pct, fg3_pct, ft_pct の3項目。合わせて予測対象14項目。

カウントは per-minute で学習する。 カウントをそのまま学習すると出場時間の分散に支配され、出場時間の予測誤差がそのまま全項目に伝播する。レートで学習し、整合化の段階で出場時間を掛ける。

成功率は per-minute にしない。 率は時間当たりの量ではないため per-minute 化は無意味である。目的変数はシュリンク済み成功率、学習重みは試投数とし、出力は clip(0.01, 0.99) に収める（ロジット変換の定義域確保）。特徴量の側も {stat}_per_min ではなくシュリンク済み率と試投数を与える。

2.4 個人スタッツの整合化

選手ごとに独立予測した値をそのまま使わない。**5人の得点合計がチームの予想スコアと一致せず、同じ画面に矛盾した数字が並ぶ。**

前提: 予測の持ち方

予測する	導出する
2FGA 3FGA FTA（試投数、カウント）	2FGM = 2FG% × 2FGA など
2FG% 3FG% FT%（成功率、0～1）	PTS = 2FGM×2 + 3FGM×3 + FTM
OR DR AS TO ST BS F FD（カウント）	TR = OR + DR、FGA = 2FGA + 3FGA

チーム側の目標値も同じ構造で持つ（prediction_team_targets、1.5）。各項目を独立に生成すると、約2%の確率で「FT成功数>FT試投数」のような実行不能な目標が生まれ、整合化が原理的に成立しない（検証済み）。

アルゴリズム

```

import numpy as np
from scipy.optimize import brentq

EPS = 1e-4
ATTEMPTS = ('fg2a', 'fg3a', 'fta')
COUNTS = ('oreb', 'dreb', 'ast', 'tov', 'stl', 'blk', 'pf', 'fd')
PCTS = (('fg2_pct', 'fg2a'), ('fg3_pct', 'fg3a'), ('ft_pct', 'fta'))

def logit(p): return np.log(p / (1 - p))
def sigmoid(z): return 1 / (1 + np.exp(-z))

def shift_to_target(pct, att, target_made):
    """ 成功率をロジット空間で一律シフトし、 $\sum(pct\_i \times att\_i) = target\_made$  を満たす。

    ロジットは  $(0, 1)$  を  $(-\infty, \infty)$  に写すため、どれだけシフトしても  $[0, 1]$  を出ない。
     $f(d)$  は  $d$  について単調増加なので、解は一意に存在する。クリップは不要。
    """
    if att.sum() <= 0:
        return pct
    lo = logit(np.clip(pct, EPS, 1 - EPS))
    f = lambda d: float((sigmoid(lo + d) * att).sum()) - target_made
    if f(-50) > 0 or f(50) < 0: # 目標が到達不能 (= チーム目標が不正)
        raise InfeasibleTargetError(target_made, float(att.sum()))
    return sigmoid(lo + brentq(f, -50, 50))

def reconcile(players, target, overtime=0, iters=3):
    """ 選手予測をチームレベル目標に整合させる。 """
    # 1. 期待出場時間
    m = np.array([p.avail_prob * p.minutes_if_plays for p in players])

    # 2. 総出場時間を正規化 (5人 × 40分。延長は +25分/回。予測時は延長を仮定しない)
    m = m * (200 + 25 * overtime) / m.sum()

    # 3. レート × 出場時間 → カウント
    x = {s: np.array([p.rate[s] for p in players]) * m
          for s in ATTEMPTS + COUNTS}
    # 成功率はレートではなくそのまま使う
    pct = {k: np.clip(np.array([p.pct[k] for p in players]), EPS, 1 - EPS)
            for k, _ in PCTS}

    for _ in range(iters):
        # 4. 試投数・その他カウントをチーム目標へ比例スケール
        for s in ATTEMPTS + COUNTS:
            tot = x[s].sum()
            if tot > 0:
                x[s] = x[s] * target[s] / tot

        # 5. 成功率をチーム目標へロジット空間シフト
        for k, a in PCTS:
            target_made = target[k] * target[a] # チームの成功数
            pct[k] = shift_to_target(pct[k], x[a], target_made)

        # 6. 得点・成功数は導出 (独立に持たない)
    made = {k: pct[k] * x[a] for k, a in PCTS}
    pts = made['fg2_pct'] * 2 + made['fg3_pct'] * 3 + made['ft_pct']
    return m, x, pct, pts

```

手順5でクリップを使ってはならない。成功率を [0, 1] の範囲で一律に掛け算すると、1.0 に張り付いた選手が出た時点で合計を目標に合わせられなくなり、反復が振動する。実測では失敗ケースの 73% が FT 成功率で発生した（pf のクリップは無関係であることを、外して再測定して確認済み）。

pf の 5.0 クリップも行わない。期待値としての反則数が5を超えることは実運用上ほぼないうえ、クリップは整合を壊す。値域は表示側で扱う。

収束と許容誤差（実測）

1,500ケースのランダム入力で測定した結果。

反復	項目誤差 中央値	99%点	最大	得点誤差 最大
1	0.0000%	3.39%	10.73%	0.00000
2	0.0000%	0.94%	3.51%	0.00000
3	0.0000%	0.27%	1.87%	0.00000

反復回数は3とする。得点誤差は反復1回でも厳密にゼロになる（手順6が恒等式による導出であり、手順5で成功数が目標に厳密一致するため）。残る誤差は手順4と手順5の相互作用による試投数側のずれで、3回で99%点が 0.27% に収まる。

許容誤差は得点 ±0.5点、その他 ±2%、制約違反0件とする（受け入れ基準 A-13）。

旧版の「試投数と成功数を別々に比例スケールし、2回反復」方式は収束しない。同一条件での実測は項目誤差の最大 24.7%、得点誤差 4.42点だった。

この手順は verification/03_reconciliation_logit.py で再検証できる。

例外時の扱い

InfeasibleTargetError はチーム目標が不正（成功数 > 試投数 など）であることを意味し、整合化の失敗ではなく上流の欠陥である。当該試合の個人スタッツ予測を破棄し、チーム予測（勝率・予想スコア）のみを保存する。画面には「この試合の個人スタッツ予測は算出できませんでした」と表示する。無言でクリップして辻褄を合わせない。

2.5 Elo の算出

K, HOME_ADVANTAGE, SEASON_REGRESSION は walk-forward の Brier を 目的関数としてグリッド探索する。以下は探索の初期値。

K ∈ {12, 16, 20, 24, 32} 初期 20
HOME_ADVANTAGE ∈ {40, 55, 70, 85} 初期 70
SEASON_REGRESSION ∈ {0.5, 0.6, 0.75, 0.85} 初期 0.65

```
ha = 0 if game.spectator_restricted else HOME_ADVANTAGE

expected_home = 1 / (1 + 10 ** ((elo_away - elo_home - ha) / 400))
actual_home    = 1 if home_score > away_score else 0

margin = abs(home_score - away_score)
multiplier = log(margin + 1) * (2.2 / (0.001 * abs(elo_home - elo_away) + 2.2))

elo_home += K * multiplier * (actual_home - expected_home)
elo_away -= K * multiplier * (actual_home - expected_home)
```

観客制限期間はホームアドバンテージを0にする。2020-21 / 2021-22 の約1,500～2,000試合が無観客または収容制限下にあり、この期間はホームアドバンテージ自体が消失することが各国リーグで確認されている。固定値のまま更新すると Elo が系統的に歪む。

HOME_ADVANTAGE の検算: Elo 差60点は勝率 58.6%、ホーム勝率60%に対応するのは 70.5点。旧版の60はオーダーは合うが実測値ではない。

シーズン間の回帰

elo_new_season = 1500 + (elo_prev_season_end - 1500) * SEASON_REGRESSION

Bリーグは NBA よりロスター変動が大きいため、NBA 標準の 0.75 より強い回帰（0.5～0.65）が適切な可能性が高い。実データで探索する。

昇格クラブ: 1500（＝旧B1平均）を与えない。B.ONE 時代の Elo があればリーグ間オフセットを推定して平行移動し、なければ前季 PREMIER 下位層の水準（1400前後）を初期値にする。 `games_played < 10` の間は Elo の信頼度を特徴量として渡す。

チーム別ホームアドバンテージ: 素朴な推定は過学習する。1チームあたりのホーム試合は年約30試合で、ホーム勝率の標準誤差は約9ポイント、見かけの差の大半はノイズである。階層モデル（部分プーリング）にする。

```
HA_team = HA_league + shrink * (HA_team_observed - HA_league)
shrink  = n_home / (n_home + k)      # k は経験ベイズで推定。典型 50～100
```

2.6 当季集計の縮約

```
winrate_shrunk = (w + k * prior) / (n + k)
```

`k = 10`、`prior` は前季勝率をシーズン間回帰させた値。「重みを下げる」という記述ではなく、この式を使う。

2.7 根拠のグループ集約

SHAP 値は個別特徴ではなく、以下のグループに合算して表示する。

group_key	含む特徴量	表示名
TEAM_STRENGTH	Elo、勝率、得失点差、SOS、ORtg/DRtg、Four Factors	チーム力
SCHEDULE	連戦、前戦結果、休養、連続アウェイ、移動距離	日程・疲労
PLAYER	欠場、出場時間喪失、出場時間集中度、ロスター構成	選手
VENUE	メイン会場、収容人数、動員	会場

個別特徴のまま表示すると、相互相関0.6～0.9のためSHAP値が分散し、ほぼ同じ条件の2試合で「チーム力の差」と「直近5試合の勝率」が入れ替わって表示される。

表示ルール

- 数値（`contribution`）を画面に出さない。方向と相対的な強さ（バーの長さ、3～4段階）で示す
- `%` は勝率専用に予約する。SHAP値はログオッズ空間であり確率への加算ではない
- `favors` で有利な側を日本語で明示する
- 生の特徴量名（`elo_diff` 等）を画面に出さない
- **選手個人の能力・資質への評価を含む表現を使わない**（可: 「出場時間の減少」/ 不可: 「不調」「衰え」）
- 動員が上位に来て因果として表示しない（強さの代理変数であるため）

表示例:

group	label_ja	value_text	favors
TEAM_STRENGTH	チーム力の差	+82ポイント	HOME
SCHEDULE	アウェイの休養	中0日（ホームは中2日）	HOME
PLAYER	アウェイの主力欠場	平均28分の選手が1名	HOME
VENUE	開催会場	代替アリーナ	AWAY

冒頭に一行の要約を置く: 「ホームのチーム力が上回っていること、アウェイが2連戦の2戦目で疲労していることが、この予測の主な理由です。」

`explainer` は推論ループの外で1回だけ構築する。

3. API 仕様

ベースパス `/api/v1`。すべて JSON。

主要導線（今日の予測・試合一覧）はAPIを経由しない。バッチが書き出した静的JSON（`/data/today.json` 等）をPagesが配信する。APIは動的クエリと書き込みに限定する。

3.1 共通

```
{ "data": { ... }, "meta": { "generatedAt": "...", "modelVersion": "...", "stale": false } }
{ "error": { "code": "NOT_FOUND", "message": "..." } }
```

コード	HTTP
<code>BAD_REQUEST</code>	400
<code>UNAUTHORIZED</code>	401
<code>NOT_FOUND</code>	404
<code>ALREADY_FINAL</code>	409
<code>RATE_LIMITED</code>	429
<code>INTERNAL</code>	500

3.2 入力検証

すべてのパラメータを境界で検証し、不一致はD1にもキャッシュにも触れず400で返す。

パラメータ	制約
<code>date</code>	<code>^\d{4}-\d{2}-\d{2}\$</code> かつ実在日付かつシーズン範囲内
<code>slug</code>	<code>^[a-z0-9-]{1,40}\$</code>
<code>gameId</code>	公式試合IDの形式
<code>limit</code>	既定20、最大100。範囲外はクランプ

範囲外の日付を弾かないと、URL空間が無限になりクロウラの総当たりで無料枠が枯渇する。

D1クエリは必ず `env.DB.prepare(sql).bind(...)` を使う。テンプレートリテラルや文字列連結でSQLを組まない。

3.3 公開エンドポイント

メソッド	パス	用途
GET	<code>/games?date=</code>	指定日の試合（シーズン範囲内のみ）
GET	<code>/games/:gameId</code>	試合詳細（試合前/試合後で内容が変わる）
GET	<code>/results?date=</code>	終了した試合と予測の対比
GET	<code>/teams /teams/:slug</code>	チーム一覧・詳細
GET	<code>/accuracy</code>	<code>accuracy_summary</code> から読む
GET	<code>/health</code>	稼働状態と当日カウンタ

GET `/games/:gameId`（試合前）

```

{
  "data": {
    "game": {
      "gameId": "...", "tipoffAt": "...", "league": "PREMIER",
      "home": { "clubId": "...", "slug": "...", "name": "...", "shortName": "..." },
      "away": { ... },
      "venue": { "name": "...", "isPrimary": true },
      "prediction": {
        "homeWinProb": 0.68,
        "predHomeScore": 84, "predAwayScore": 78,
        "isProvisional": false, "isFinal": false, "isEarlySeason": false,
        "modelVersion": "winner-v1.0.0",
        "summary": "ホームのチーム力が上回っていること、アウェイが2連戦の2戦目で疲労していることが、この予測の主な理由です。",
        "reasons": [
          { "group": "TEAM_STRENGTH", "label": "チーム力の差",
            "value": "+82ポイント", "favors": "HOME", "strength": 3 }
        ]
      }
    }
  },
  "playerPredictions": [
    {
      "playerId": "...", "name": "...", "position": "PG", "clubId": "...",
      "availProb": 0.95,
      "summary": { "min": 31.2, "pts": 18.4, "reb": 3.1, "ast": 6.1 },
      "error": { "min": 5.8, "pts": 4.8, "reb": 2.1, "ast": 1.5 },
      "box": {
        "fg": { "m": 6.4, "a": 13.1, "pct": 0.489 },
        "fg2": { "m": 4.1, "a": 7.8, "pct": 0.526 },
        "fg3": { "m": 2.3, "a": 5.3, "pct": 0.434 },
        "ft": { "m": 3.3, "a": 3.9, "pct": 0.846 },
        "oreb": 0.6, "dreb": 2.5,
        "ast": 6.1, "tov": 2.2, "stl": 1.1, "blk": 0.3,
        "pf": 2.4, "fd": 3.1,
        "efgPct": 0.577, "tsPct": 0.601
      }
    }
  ],
  "recentForm": { "home": { "last5": ["W","W","L","W","W"], "avgMargin": 6.2, "elo": 1582 },
    "away": { ... } },
  "modelAccuracy": { "version": "winner-v1.0.0", "accuracy": 0.682, "brier": 0.204, "n": 312 }
}

```

`availProb < 0.5` の選手は `playerPredictions` に含めない。`strength` は1~4の段階値で、SHAPの生値は返さない。

`pct` は試投数の予測値が閾値以上のときのみ返す。閾値未満は `null` とし、クライアントは分数のみを表示する。

項目	閾値
FG%	<code>fg.a >= 4</code>
2FG% / 3FG%	それぞれの <code>a >= 3</code>
FT%	<code>ft.a >= 3</code>
EFG% / TS%	<code>fg.a >= 4</code>

`plusMinus` の予測は返さない。試合後のレスポンスでは実績値のみを `actual` 側に含める。

`pts` と `reb` はサーバ側で導出して `summary` に入れる（クライアントで計算させると実装ごとにずれる）。`box` には導出元のカウントをそのまま入れ、クライアントは表示するだけにする。

GET /games/:gameId (試合後)

```
{
  "data": {
    "game": { ..., "status": "FINISHED", "homeScore": 88, "awayScore": 81 },
    "prediction": { "homeWinProb": 0.68, "predHomeScore": 84, "predAwayScore": 78,
      "isFinal": true },
    "evaluation": {
      "isCorrect": true,
      "scoreError": 3,
      "bucketContext": { "bucket": "60-70%", "n": 42, "correct": 29, "rate": 0.690 }
    }
  }
}
```

`bucketContext` は「68%と予想した試合は42試合中29試合が的中」という表示の根拠になる。**外れた試合でもこれを返す。**

GET /accuracy

`accuracy_summary` を単一テーブルから読む（読み取り数十行）。

```
{
  "data": {
    "overall": { "accuracy": 0.682, "brier": 0.204, "n": 312, "baselineAccuracy": 0.601 },
    "bySeason": [ ... ],
    "byModel": [ ... ],
    "calibration": [ { "bucket": "60-70%", "predicted": 0.65, "actual": 0.63, "n": 88 } ],
    "byProvisional": { "provisional": { "accuracy": 0.66, "n": 120 },
      "confirmed": { "accuracy": 0.70, "n": 192 } }
  }
}
```

GET /health

```
{
  "data": {
    "jobs": [ { "job": "daily_ingest", "lastSuccessAt": "...", "lastRunStatus": "SUCCESS" } ],
    "stale": false,
    "activeModels": ["winner-v1.0.0"],
    "quota": { "workersRequestsToday": 1240, "d1RowsReadToday": 98000,
      "d1RowsWrittenToday": 3200, "cacheHitRate": 0.91 },
    "degraded": false
  }
}
```

`error_message` を返さない。詳細は返さず、状態のみを公開する。

3.4 内部エンドポイント

`Authorization: Bearer` を要求する。トークンは用途で分離する。

メソッド	パス	トークン
POST	<code>/internal/games</code> <code>/internal/stats</code> <code>/internal/entries</code> <code>/internal/ratings</code>	INGEST_TOKEN
POST	<code>/internal/predictions</code>	INGEST_TOKEN
POST	<code>/internal/finalize</code>	FINALIZE_TOKEN（破壊的操作のため分離）
POST	<code>/internal/evaluate</code> <code>/internal/summary</code> <code>/internal/log</code>	INGEST_TOKEN

`/internal/predictions` の必須ガード

- Bearer を定数時間比較する（両者を SHA-256 ハッシュ化してから比較する。Workers の `timingSafeEqual` は長さ不一致で例外を投げる）
- Bearer プレフィックスを正規表現で厳格に検証する
- Zod で型と値域を検証する
- `games.tipoff_at <= now` の試合は 409 ALREADY_FINAL で拒否する
- 旧行の非活性化と新行の挿入を単一 `batch()` で実行する
- UPDATE には必ず `WHERE is_final = 0` を付ける
- 日次の書き込み回数上限（予測は1日2000件）を設け、超過時は 429

バッチサイズ（テーブルごとに算出する）

D1 Free には1 Worker 呼び出しあたり50クエリという上限がある（Paid は1,000）。旧版の「1リクエスト最大500件」は、この上限を9テーブル中6つで超えていた。

1文に詰められる行数は `floor(100 / 列数)`（バインドパラメータ上限100）で決まるため、

```
max_rows_per_request = floor(100 / 列数) × 40
```

とする（係数40は、50クエリのうち10を非活性化・ログ・整合性確認などの余裕として残すため）。

テーブル	列数	1文の行数	1リクエスト上限	旧設計	旧設計での必要文数
<code>player_predictions</code>	26	3	120	500	167（上限の3.3倍）
<code>player_game_stats</code>	22	4	160	200	50
<code>games</code> / <code>team_game_stats</code>	20	5	200	500	100
<code>predictions</code>	18	5	200	500	100
<code>prediction_team_targets</code>	17	5	200	—	—
<code>game_entries</code> / <code>team_ratings</code>	6-7	14-16	560-640	500	32-36

上限値は `api/src/config/batch-limits.ts` に定数として置き、列数から機械的に算出する関数と、その結果が50クエリ以内であることを検証するテスト（`test_batch_size_within_query_limit`、A-10）を置く。列を1つ増やしたときに上限を静かに超えることを防ぐ。

`batch()` 内の各文が50クエリ制限にどう計上されるかは、Cloudflare の公式ドキュメントに記載がない。1文=1クエリという最も厳しい前提で設計し、Phase 0（P0-13）で実アカウントでの実測を行う。実測の結果 `batch()` 全体が1クエリと数えられるのであれば上限を緩められるが、**実測前に緩めない。**

CPU 時間は制約ではない。実測では500行の JSON（141KB）のパーズ 0.56ms、検証 0.28ms、`batch()` 組立 0.50ms で合計 1.34ms。Zod で3〜4ms と見込まれるが、Workers Free の CPU 10ms に対して余裕がある。**バッチサイズは CPU ではなくクエリ数で決める。**

トークンの2キー方式: 認証は `INGEST_TOKEN` と `INGEST_TOKEN_NEXT` のいずれかに一致すれば通す。無停止で回転できる。

3.5 キャッシュとレート制限

パス	Cache-Control
/games?date= (過去日)	public, max-age=60, s-maxage=3600, stale-while-revalidate=86400
/games/:id (確定済み)	同上
/games/:id (未実施)	public, max-age=60, s-maxage=300, stale-while-revalidate=3600
/accuracy	public, max-age=60, s-maxage=3600, stale-while-revalidate=86400
/teams*	public, max-age=60, s-maxage=3600
/health	no-store

max-age を長く取らない。一度 `max-age=86400` を返すとそのクライアントは24時間必ず古い値を見る。エッジをページしても届かない。

「ページする」に依存しない。Workers の Cache API の `delete()` はそのコロにしか効かず、`*.pages.dev` ではゾーンページも使えない。**キャッシュキーにデータ世代（最終取り込み時刻）を含める**ことで、新しい世代が出れば自動的に別キーになり、ページ自体が不要になる。

キャッシュキーは許可リストにないクエリパラメータを除去して正規化する。`?fbclid=` のようなパラメータで無限にフラグメント化すると、キャッシュを迂回して D1 に直撃する（キャッシュバusting）。

レート制限

Cloudflare WAF のレトリミット（無料プラン）は次の制約を持つ。旧版の「60req/分を超えたら1分ブロック」は無料枠では設定できない。

項目	無料プランの制限
ルール数	1
カウント期間	10秒のみ（1分・5分などは有料）
ブロック時間	10秒のみ
カウント対象	IP のみ（ヘッダやクッキー単位は有料）
照合フィールド	Path と Verified Bot のみ

したがって設定は次のとおりとする。

```
式: (http.request.uri.path contains "/api/v1/") and (not cf.bot_management.verified_bot)
しきい値: 10秒あたり 30 リクエスト
動作: ブロック 10秒
```

10秒ブロックは総量規制としては弱い。攻撃者は10秒待てば再開できる。これは「誤った実装のクライアントによる暴走」と「素朴なクローラの総当たり」を止めるための措置であり、意図的な攻撃には効かない。真の防御は次の2つで、WAF はその補助と位置づける。

- 1. 主要導線が静的配信で Workers を通らない（枠が枯渇しても当日の予測は表示され続ける）
- 2. URL 空間の有限化（シーズン範囲外の日付は D1 到達前に404）

Workers 側にも Cache API を使った簡易カウンタを置き、`429 + Retry-After` を返す。これは WAF より柔軟な期間を取れるが、Cache API はコロ単位であるため厳密な総量規制にはならない。それを承知のうえで、階層の1つとして持つ。

CORS はレート制限の代替ではない。ブラウザ内の制限であり、サーバからのアクセスには効かない。しかもフロントは静的出力でサーバサイド fetch するため `Origin` すら付かない。目的が「他サイトからの埋め込み抑止」であることを明記する。

3.6 セキュリティヘッダ

`web/public/_headers` に設定する。

```
/*
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: geolocation=(), camera=(), microphone=()
Content-Security-Policy-Report-Only: default-src 'self'; script-src 'self' 'sha256-<テーマ適用スクリプトのハッシュ>'; style-src 'self' 'unsafe-inline'; img-src 'self' data;
```

テーマ適用の inline script は**最初から sha256- ハッシュ指定で許可する形で書く**。後から CSP を入れるとこのスクリプトが真っ先に壊れ、「CSPを入れたらテーマがちらつくのでCSPをやめる」という後退が起きる。Report-Only で数週間様子を見てから強制的に切り替える。

`dangerouslySetInnerHTML` を使わない。

4. バッチ処理詳細

4.1 ワークフロー定義

```
# .github/workflows/daily-ingest.yml
name: daily-ingest
on:
  schedule:
    - cron: '0 21 * * *' # 06:00 JST
  workflow_dispatch:
permissions:
  contents: write # 静的JSONとスナップショットをコミットするため
                  # 他のワークフローは contents: read のまま
concurrency:
  group: dl-write # D1 へ書き込むジョブの同時実行を禁止する
  cancel-in-progress: false # 走行中のジョブは殺さず、後続を待たせる
jobs:
  ingest:
    runs-on: ubuntu-latest
    timeout-minutes: 30
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.12', cache: 'pip' }
      - run: pip install -r batch/requirements.txt
      - run: python -m batch.jobs.daily_ingest
    env:
      INGEST_TOKEN: ${ secrets.INGEST_TOKEN }
      API_BASE_URL: ${ vars.API_BASE_URL }
```

ワークフロー	cron (UTC)	JST	タイムアウト	permissions	concurrency
ci	push / PR	—	10分	contents: read	ci-\${ github.ref }
daily-ingest	0 21 * * *	06:00	30分	contents: write	dl-write
gameday-update	0 2,4,7 * * * (各 スロットで tipoff > now+30分 のみ対象)	11:00 / 13:00 / 16:00	20分	contents: write	dl-write
parser-canary	0 22 * * *	07:00	10分	contents: read	canary
monthly-train	0 19 1 * *	毎月2日 04:00 (1日 19:00 UTC は JST では 翌日04:00)	120分	contents: read	dl-write
backfill	手動のみ	—	300分	contents: read	dl-write

`contents: write` を与えるのは `daily-ingest` と `gameday-update` の2つだけ。この2つは静的JSON（`web/public/data/`）とスナップショット（`batch/snapshot/`）をリポジトリへコミットするため書き込み権限が要る。`monthly-train` はモデルを D1 に登録するだけでコミットしないため `read` のままでよい。残りも同様。既定を `read` に置き、必要なものだけ個別に緩めるという順序を崩さない。

`concurrency: dl-write` で D1 へ書き込むジョブの同時実行を禁止する。`cancel-in-progress: false` とし、走行中のジョブを殺さず後続を待たせる。

この設定がないと、cron 遅延で `daily-ingest` (21:00 UTC・遅延して02:00に開始) と `gameday-update` (02:00 UTC) が重なった場合に、**両方が同じ試合に対して再推論を行い、`revision` の採番が競合する**。D1 にはリクエストを跨ぐトランザクションがないため、「最大 `revision` を読む → +1 して挿入する」という2段階が Read-Modify-Write 競合を起こし、`UNIQUE (game_id, model_version, revision)` 違反か、より悪い場合は2本の `is_active = 1` が同時に立つ（部分ユニークインデックスがあるため後者は防がれ、片方が例外で落ちる）。**走行中を殺してはならないのは、殺されたジョブが `batch()` の途中で止まり `ingestion_logs` が `RUNNING` のまま残るため**。

`finalize` は Workers の Cron Trigger (毎時) で `/internal/finalize` を叩く。GitHub Actions ではなく Workers に置くのは、外部アクセスに依存しない純粋な D1 操作であり、スクレイピングの失敗に巻き込まれてはならないため。`finalize` は `predictions` の UPDATE のみを行い、`d1-write` グループには入らない（毎時実行を待たせると `freeze` が遅れるため）。`freeze` は他の書き込みと衝突しない（対象行が `is_final = 0 → 1` の遷移のみ）。

cron は UTC。GitHub Actions の cron は数十分～数時間遅延しうるため、遅延しても壊れない設計（tipoff ガード）を前提とする。`gameday-update` のスロットは JST 11:00 / 13:00 / 16:00 で、B.LEAGUE の主な開始時刻（14:05 / 15:05 / 17:05 / 19:05）の2～3時間前を捕捉する。

4.2 daily_ingest

```

def run():
    log = start_log("daily_ingest")
    status = "SUCCESS"

    # 0. robots.txt / 利用規約の変更検知
    if robots_changed() or terms_changed():
        finish_log(log, "ABORTED", "robots or terms changed")
        sys.exit(1)

    # 1. スクレイピング区間 (ここだけ AbortedByRemote を捕捉する)
    try:
        games = scrape_finished_games(yesterday_jst())
        post_games(games)
        for g in games:
            # 3秒間隔・値域検証
            post_stats(scrape_boxscore(g.id))
    except AbortedByRemote:
        status = "PARTIAL"
        # 内部処理は継続する
    except ParseErrorStreak:
        status = "PARTIAL"

    # 2. スナップショット更新 (D1 へ書いたのと同じデータから書き出す)
    write_snapshot()
    # batch/snapshot/*.parquet + MANIFEST.json

    # 3. 内部処理 (外部アクセスの失敗に巻き込まれない)
    detect_score_revisions()
    # result_revision を進め再評価キューへ
    evaluate_finished_games()
    # VOID を除外して照合
    rebuild_accuracy_summary()
    recompute_ratings(from_date=affected_min_date())

    # 4. 推論 (未開始試合のみ)
    ds = load_snapshot()
    # D1 は読まない
    for g in get_upcoming_games(days=7):
        # tipoff_at > now のみ
        feats = build_features(g.id, as_of=g.tipoff_at, ds=ds)
        p = predict(feats)
        # 勝率・margin・total
        target = predict_team_rates(feats)
        # 整合化の目標値 (試投数+成功率)
        box = reconcile(predict_players(feats), target)
        # 2.4
        post_prediction(g, p, target, box, data_as_of=ds.max_finished_at)

    # 5. 静的JSON の書き出し
    write_static_json()
    commit_and_push(["web/public/data/", "batch/snapshot/"])

    finish_log(log, status)
    if status == "PARTIAL":
        sys.exit(1)
        # Actions の失敗通知に乗せる

```

AbortedByRemote の捕捉範囲をスクレイピング区間に限定する。旧版は try が処理全体を覆っていたため、前日結果が取れないだけで freeze・照合・Elo・推論まですべてスキップされ、その日の予測更新が止まっていた。

PARTIAL を exit 1 で終える。exit 0 だと Actions の失敗通知に乗らず、誰も気づかないまま数日放置される。

4.3 スクレイピングクライアント

```

class RateLimitedClient:
    MIN_INTERVAL = 3.0      # 秒。短縮しない
    JITTER = 1.0
    TIMEOUT = 30
    MAX_RETRY = 2
    MAX_BYTES = 5 * 1024 * 1024
    DAILY_LIMIT = 3000
    UA = "<name>/1.0 (+https://<domain>/about)"

    def get(self, url: str) -> str:
        self._check_daily_limit()
        self._wait()
        with httpx.stream("GET", url, headers={"User-Agent": self.UA},
                          timeout=self.TIMEOUT) as r:
            if r.status_code in (429, 503):
                raise AbortedByRemote(url, r.status_code)
            r.raise_for_status()
            buf = bytearray()
            for chunk in r.iter_bytes():
                buf += chunk
            if len(buf) > self.MAX_BYTES:
                raise ResponseTooLarge(url)
            return buf.decode(r.encoding or "utf-8")

```

Open-Meteo は使用しない（天候は見送り項目）。仮に外部APIを使う場合も、公式サイト向けの3秒間隔を無条件に適用しない。

4.4 パーサと値域検証

```

class ParseError(Exception):      """必須項目が抽出できない。構造変更の可能性"""
class ValidationError(Exception): """値域外。当該試合をスキップ"""

def parse_boxscore(html: str) -> BoxScore:
    ...                          # 必須項目が取れなければ ParseError

def validate(box: BoxScore, venue: Venue) -> None:
    check(0 <= box.pts <= 250)
    check(box.fgm <= box.fga)
    check(all(0 <= p.minutes <= 60 for p in box.players))
    if box.attendance and venue.capacity:
        check(box.attendance <= venue.capacity * 1.2)

```

数値パースは全角数字、カンマ、 "-"（未出場）、"MM:SS" 形式、"DNP"、空文字を考慮する。

`ParseError` が連続3件でジョブを中止し `PARTIAL` で記録する。修正時はサンプルを取得して合成 fixture に落としてからテストを書く。本番アクセスで試行錯誤しない。

4.5 学習ジョブ

```

def monthly_train():
    run_leakage_tests()          # 失敗したらここで停止
    verify_snapshot_manifest()   # 行数と SHA256 の照合。不一致なら学習を中止

    df = load_training_data()    # batch/snapshot/*.parquet のみを読む。D1 は読まない
    df = apply_sample_weights(df) # 時間減衰 + 観客制限期間の調整
    X, y, w = build_matrix(df)

    # 入れ子の時系列分割
    metrics, best_iters = walk_forward_validate(

```

```

X, y, w, by="season_id", max_folds=5)

# 最終モデルは best_iteration の中央値で固定学習
model = train_lightgbm(X, y, w, params=PARAMS,
                       num_boost_round=median(best_iters))

# 較正は out-of-fold で判断・fit
if needs_calibration(metrics.oof_ece):
    calibrator = fit_platt(metrics.oof_pred, metrics.oof_y)
else:
    calibrator = None

if not passes_criteria(metrics, model):
    log("基準未達のため新モデルを有効化しない", metrics)
    return

artifact = model.save_model() # テキスト形式
if len(artifact.encode()) > 1_572_864: # 1.5 MiB
    log("artifact がサイズ上限を超えたため登録しない", size=len(artifact))
    sys.exit(1) # PARTIAL 扱い。現行モデルを継続
version = register_model(artifact, calibrator, metrics) # artifact_text を D1 へ
activate_model(version) # uq_model_active が整合を保証

COUNT_STATS = ('fg2a', 'fg3a', 'fta', 'oreb', 'dreb', 'ast', 'tov', 'stl', 'blk', 'pf', 'fd')
PCT_STATS = (('fg2_pct', 'fg2m', 'fg2a'), ('fg3_pct', 'fg3m', 'fg3a'),
              ('ft_pct', 'ftm', 'fta'))

def train_player_models():
    """選手モデル。カウント11 + 成功率3 + 出場2 = 16本。学習時間の管理が要る。"""
    df = load_player_training_data() # 約20万行。スナップショットから
    X = build_player_matrix(df)

    train_and_register('PLAYER_AVAIL', X, y=df.played, folds=3)
    train_and_register('PLAYER_MIN', X[df.played], y=df.minutes, folds=3)

    for stat in COUNT_STATS: # 11項目
        # per-minute レートで学習する。カウントで学習すると出場時間の分散に支配され、
        # 出場時間の予測誤差がそのまま全項目に伝播する
        y = df[stat] / df.minutes
        train_and_register('PLAYER_RATE', X[df.minutes > 0], y=y,
                           target=stat, folds=3)

    for pct, made, att in PCT_STATS: # 3項目
        # 率は per-minute にしない（時間当たりの量ではない）。
        # 目的変数はシュリンクした成功率、重みは試投数。
        k, prior = SHRINK_K[pct], league_mean(made, att)
        y = (df[made] + k * prior) / (df[att] + k)
        train_and_register('PLAYER_RATE', X[df[att] > 0], y=y,
                           target=pct, weight=df[att], folds=3,
                           params={**PARAMS_REG, "objective": "regression"})

def train_team_rate_models():
    """整合化の目標値。選手側と同じ「試投数 + 成功率」の構造で予測する。"""
    for stat in COUNT_STATS:
        train_and_register('TEAM_RATE', Xt, y=team_df[stat], target=stat, folds=5)
    for pct, made, att in PCT_STATS:
        train_and_register('TEAM_RATE', Xt, y=team_df[made] / team_df[att],

```

```
target=pct, weight=team_df[att], folds=5)
```

TEAM_RATE を「成功数」で学習しない。チーム目標が 成功数 > 試投数 を満たしうる状態だと、整合化は原理的に成立しない（2.4）。試投数と成功率で学習すれば構造的に保証される。

成功率モデルの出力は `clip(0.01, 0.99)` に収める。ロジット変換の定義域を確保するため。

artifact_text のサイズを登録前に検査する。1.5MB を超えたら登録せず、現行モデルを継続使用して `exit 1` で終える。`num_boost_round` の上限を 1,200 とし、early stopping が効かずに上限まで回るケースを防ぐ。

選手モデルの fold 数は3に制限する（チームモデルは5）。選手16本 + チーム14本の学習が必要で、タイムアウト120分に収めるための措置である。

学習時間が猝に収まらない場合の対処は、この順で行う。

1. 木をさらに小さくする（`num_leaves 4`、`max_depth 2`）
2. 学習行数を直近5シーズンに制限する
3. 低レート項目（`stl blk`）を「その選手の直近平均」で代替し、モデルを持たない

3を選ぶ場合、画面には「この項目は直近平均を表示しています」と明示する。予測と称して平均を出すことはしない。

実測は Phase 0（P0-10）で確認する。

検証セットの分離

```
fold i:  train = seasons[:i-1]
         valid = seasons[i-1]      ← early stopping 用
         test  = seasons[i]        ← 一切触れない
```

旧版は `train_lightgbm(X, y)` に検証セットがなく、early stopping が成立していなかった（学習データを渡せば止まらず、評価シーズンを渡せばリーク）。

4.6 採用基準

```
MIN_EVAL_N    = 500      # これ未満では一切比較しない
MIN_EFFECT    = 0.003    # 最小実質差 (Brier)
ECE_FLOOR_K   = 1.5      # ノイズフロア95%点に対する倍率

def passes_criteria(m, model) -> bool:
    # 0. サンプル数の下限。満たさなければ比較そのものを行わない
    if m.n < MIN_EVAL_N:
        log("n < 500 のため比較せず現行モデルを継続", n=m.n)
        return False

    current = current_active_brier_on_same_window()
    return (
        # 1. 同一の評価ウィンドウで再評価した Brier が良い
        m.brier < current
        # 2. 有意 かつ 実質的
        and m.brier_diff_ci_upper < 0          # ブートストラップCIが0を跨がない
        and (current - m.brier) >= MIN_EFFECT # 点推定の差が 0.003 以上
        # 3. ベースラインを上回る
        and m.brier < m.baseline_elo_brier
        # 4. 較正 (n 依存のノイズフロア。固定値 0.05 を使わない)
        and m.ece < ece_noise_floor(m.pred_probs, m.n, q=0.95) * ECE_FLOOR_K
        # 5. 特徴量の健全性
        and max(m.feature_null_rates.values()) <= 0.30
    )
```

Accuracy を採用ゲートに入れない。ホーム勝率60%の下では良い確率モデルでも Accuracy はベースラインとほぼ並ぶため、Brier で明確に優れているモデルが誤って棄却される。Accuracy は公開指標としてのみ使う。

`n < 500` では比較を行わない。実測では、理想モデル（真の勝率を知る上限）とベースラインの Accuracy 差の標準偏差が `n=200` で 0.041 あり、旧基準

の ± 0.03 はノイズと区別できなかった。ECE も同様で、完全校正モデルの ECE 95%点は $n=200$ で 0.1163、 $n=500$ で 0.0734、 $n=1,200$ で 0.0479 と n に強く依存する。固定閾値 0.05 は $n=200$ では良いモデルを落とし、 $n=1,200$ ではほぼ常に素通りする。

`ece_noise_floor()` はモデル自身の予測確率分布からブートストラップで毎回推定する。定数表を持たない。実装と検証は `verification/01_ece_noise_floor.py` を流用する。

月次再学習の検出力は低いが、それは受け入れる。真の差 0.03 を検出できる確率は $n=200$ で 12.7%、 $n=1,200$ でも 54.8% にとどまる。上記のゲートは「ノイズで誤って差し替える」ことを防ぐ側に倒してあり、その代償として「実際の改善を見逃す」。見逃した改善は次回以降の月次で拾う。検出力を上げるためにゲートを緩めてはならない。

比較は同一の評価ウィンドウで行う。`model_versions.cv_brier`（学習当時の値）を直接比較しない。新旧で検証対象のシーズンが違えば比較になっていない。

欠損率チェックが必要な理由: 新特徴量を追加した際、元データが未投入で大半が None になっても、Brier が偶然わずかに良ければ従来の基準では採用されてしまう。実際には新特徴量が機能していないどころか既存特徴量の重みが劣化しており、数値がすべて基準内なので誰も気づかない。

4.7 LightGBM パラメータ

```
PARAMS = {
    "objective": "binary",
    "metric": ["binary_logloss"],
    "learning_rate": 0.03,
    "num_leaves": 7,          # 8000行に対し15は過大
    "max_depth": 3,
    "min_data_in_leaf": 100,
    "feature_fraction": 0.7,
    "bagging_fraction": 0.8,
    "bagging_freq": 1,
    "lambda_l2": 1.0,
    "num_boost_round": 1200,  # 上限。artifact サイズが 1.5MB を超えないための制約でもある
    "early_stopping_round": 100,
    "verbosity": -1,
    # 再現性。これを欠くと Brier が  $\pm 0.003$  ぶれ、採用判定が乱数になる
    "seed": 42,
    "bagging_seed": 42,
    "feature_fraction_seed": 42,
    "deterministic": True,
    "force_row_wise": True,
    "num_threads": 4,
}
```

探索範囲: `num_leaves` $\in \{4, 7, 15\}$ 、`max_depth` $\in \{2, 3, 5\}$ 、`min_data_in_leaf` $\in \{40, 100, 200\}$ 、`learning_rate` $\in \{0.02, 0.05\}$ 。

精度が出ないときに木を深くしない。この問題の信号量（Elo差だけで Brier 0.215 程度、上限でも 0.20 前後）に対し、深さ5・葉15は5次の交互作用を許す設定で過大である。まず特徴量を見直す。

`num_leaves` を 15 にする変更は、artifact サイズの再測定とセットでのみ許可する。書式からの推定では `num_leaves=15` / 木1,500本で 2.56MB となり、D1 の1行上限（2,000,000バイト）を超える。サイズガードが登録を拒否するため事故にはならないが、「探索範囲に 15 があるから」という理由で無自覚に踏むことのないよう、ここに明記する。

4.8 backfill

再開可能にする。

```
def backfill(season_id: str):
    done = fetch_ingested_game_ids(season_id)  # D1 から取得済みIDを取る
    for g in schedule_of(season_id):
        if g.id in done:
            continue  # スキップ
    ...
```

1シーズンあたりの所要時間は Phase 0（P0-4）で実測する。1試合あたり2〜4ページなら1シーズン50〜100分、10シーズンで実質10日かかる。1日1

シーズンに限る（D1 書込10万行/日の枠もある）。

再開可能にしないと、55分経過時点で 429 を食らった際に翌日また全ページを取り直すことになり、相手サイトへの負荷を二重にかける。

5. 画面詳細

5.1 コンポーネント構成

```
components/
├─ prediction/
│   └─ GameCard.tsx / GameCardCompact.tsx
│   └─ ProbabilityBar.tsx / ScoreLine.tsx
│   └─ ReasonList.tsx / PlayerStatTable.tsx
│   └─ StatusBadge.tsx / StaleBanner.tsx
│   └─ ResultComparison.tsx      試合後の対比
├─ charts/
│   └─ AccuracyTrend.tsx / CalibrationChart.tsx    インラインSVG
└─ ui/
    └─ Card.tsx / Tabs.tsx / ThemeToggle.tsx / EmptyState.tsx / Footer.tsx
```

5.2 マークアップ規約

意味構造を持たせる。旧版のモックは見出し要素が一つもなく、タブが `span`、装飾目的で `<s>` `<i>` `<em>` を使っていた。

要素	用途
<code><article></code>	試合1件
<code><h2></code>	日付見出し、対戦名（視覚的に隠してよい）
<code><h3></code>	「なぜこの予測になったか」「個人スタッツ予測」
<code><nav></code> + <code><a></code>	タブ。最低高さ44px、 <code>aria-current="page"</code>
<code><button aria-pressed></code>	テーマ切替
<code></code> + <code>class</code>	装飾。 <code><s></code> <code><i></code> <code></code> を装飾目的で使わない

確率ブロックは `role="img"` と一文の `aria-label` を持ち、内部の数値とバーは `aria-hidden="true"` にする。両方に読み上げ対象があると「68 32 ホーム68パーセント…」と重複する。

```
aria-label="ホーム <チーム名> の勝率68パーセント、アウェイ <チーム名> の勝率32パーセント。予想スコア 84対78"
```

`:focus-visible` のアウトライン色をトークン化する（`--accent` の上に `--accent` のリングが乗ると消える）。

5.3 主要コンポーネント

ProbabilityBar

- `--accent` で塗り、残りを `--track`
- 両チームの勝率を必ず表示する。圧縮カードでも省略しない
- 45～55% は「ほぼ互角」と併記する

GameCardCompact

```
ホームB 55% – 45% アウェイB
ほぼ互角  ・ 17:05
```

片側のみの表示は要件8.3違反。55%と81%が同じ見た目になると、このツールで最も伝えるべき差が失われる。

ReasonList

- 要因グループ単位（4～5件）
- 「ラベル / 値 / 有利な側」の3要素
- 数値の代わりに3～4段階のバーで強さを示す
- 冒頭に一行の要約文

PlayerStatTable

段階開示とする。20項目 × 最大10人を一度に出すとモバイルでは読めない。

初期表示（1行1選手、4項目）

選手A	PG	31.2分	18.4	3.1	6.1	▼
選手B	C	28.5分	14.2	9.8	1.4	▼

行をタップして展開（フルボックススコア）

選手A	PG	▲				
出場時間	31.2分	±5.8				
得点	18.4	±4.8				
FG	6.4 / 13.1	(48.9%)				
2P	4.1 / 7.8	(52.6%)				
3P	2.3 / 5.3	(43.4%)				
FT	3.3 / 3.9	(84.6%)				
リバウンド	3.1	±2.1	(OR 0.6 / DR 2.5)			
アシスト	6.1	±1.5				
ターンオーバー	2.2					
スティール	1.1					
ブロック	0.3					
ファウル	2.4					
被ファウル	3.1					
EFG%	57.7	・	TS%	60.1		

規約

- 初期表示は min / pts / reb / ast の4項目のみ
- 誤差（±）は主要4項目にのみ併記する。全項目に出すと画面が埋まる
- 率は分数と併記する。単独で 48.9% と出さない
- 試投数が閾値未満なら率を出さず分数のみ（1.2 / 2.4 のように）
- 展開状態は localStorage に保存しない（毎回たたんだ状態で開く）
- 展開の開閉はアニメーション付きの <details> 相当にし、キーボードで操作できるようにする
- +/- は試合前には表示しない。試合後の結果画面でのみ実績を出す
- ST / BS は誤差が平均値と同水準であることを、詳細画面の注記で1行説明する

StatusBadge / StaleBanner

状態	文言	トークン
暫定	暫定	--text-3 の枠線
確定	確定	--accent-bg
序盤	序盤のため精度が安定しません	--warn-bg / --warn
更新遅延	（全幅バナー）	--warn-bg / --warn

ResultComparison（試合後）

実際のスコア	88 - 81 (ホーム勝利)
予測	ホーム 68% / 予想スコア 84 - 78
判定	予測どおりでした (得点差の誤差 3点)
位置づけ	68%と予想した試合は、これまで42試合中29試合 (69.0%) が的中しています

外れた場合も同じ構成で、判定の文言だけが変わる。赤などの強い否定色を使わない。

5.4 テーマ実装

```
:root {
  --bg:#F6F7F9; --surface:#FFFFFF; --border:#E5E8ED;
  --text:#0F1622; --text-2:#5A6577; --text-3:#667184; --text-4:#8E99AB;
  --accent:#416A0E; --accent-bg:rgba(65,106,14,.10); --track:#E8EBF0;
  --warn:#7A4A12; --warn-bg:#FEF3E7;
}
:root[data-theme="dark"] {
  --bg:#0B0E13; --surface:#131820; --border:#212A36;
  --text:#E6EAF2; --text-2:#93A0B5; --text-3:#7C8798; --text-4:#59657A;
  --accent:#A3E635; --accent-bg:rgba(163,230,53,.12); --track:#232C3A;
  --warn:#E0A458; --warn-bg:rgba(224,164,88,.12);
}
```

```
<script>
  try { var t = localStorage.getItem('theme');
    if (t) document.documentElement.setAttribute('data-theme', t); } catch (e) {}
</script>
```

- 既定はライト。OS設定への追従は行わない
- `localStorage` へのアクセスは必ず `try/catch` で囲む
- この inline script のハッシュを CSP に登録する (3.6)

5.5 数値表示

すべての数値に `font-variant-numeric: tabular-nums` を適用する。

値	表記
勝率	整数の百分率 (68%)
予想スコア	整数 (84 - 78)。MAE 8〜10点に対し小数第1位は精度の誤認を招く
個人スタッツ (カウント)	小数第1位 (18.4)。期待値なので整数にしない
個人スタッツ (成功/試投)	6.4 / 13.1 の分数。率は括弧で併記
率 (FG%等)	小数第1位の百分率 (48.9%)。 試投数が閾値未満なら表示しない
出場時間	小数第1位+単位 (31.2分)
的中率	小数第1位+ 母数 (68.2% (312試合))
Brier	0.204 (小数点前のゼロを省略しない)。的中率ページの折りたたみ内のみ

5.6 データ取得

```
// app/page.tsx - 静的出力。ISR を使わない
export const dynamic = 'force-static';

export default function Page() {
  // ビルド時に埋め込まず、クライアントで静的JSONを読む
  // （パッチが更新しても再ビルド不要にするため）
}
```

```
const res = await fetch('/data/today.json', { cache: 'no-store' });
```

`/data/today.json` は Pages の静的アセットであり、Workers も D1 も消費しない。取得失敗時は `EmptyState` のエラー表示にフォールバックする。

Next.js 16 での `params`

`params` と `searchParams` は Promise である。同期アクセスは型エラーになる。

```
// app/schedule/[date]/page.tsx
export async function generateStaticParams() {
  return listDatesInRecentSeasons(5).map((date) => ({ date })); // 直近5シーズンのみ
}

export default async function Page({
  params,
}): {
  params: Promise<{ date: string }>; // Promise。同期で受け取らない
}) {
  const { date } = await params;
  if (!isWithinRecentSeasons(date, 5)) notFound();
  ...
}
```

静的生成の範囲

`generateStaticParams` は直近5シーズンの範囲のみを生成し、範囲外は `notFound()` を返す。Pages の1デプロイあたりファイル数上限は 20,000 で、1 ルートにつき HTML と RSC ペイロードの2ファイルが出るため、全10シーズンでは 20,184 ファイルとなり超過する（直近5シーズンなら 10,184、51%）。

CI で `out/` のファイル数を数え、18,000 を超えたら fail させる（A-14）。

```
find web/out -type f | wc -l # 18000 以下であること
```

1ルートあたりのファイル数は Next.js のバージョンで変わるため、この検査を省略しない。実測は Phase 0（P0-14）で行う。

5シーズンより古い日付・試合は静的生成せず、Workers API からクライアント fetch で表示する。

6. テスト設計

6.1 リーク検証（最重要）

検証の向きは「DBを攪乱する」。

```

def test_target_game_mutation_does_not_change_features(db, game):
    """対象試合の結果を攪乱しても特徴量が変わらないこと。これが本命。"""
    f1 = build_features(game.id, as_of=game.tipoff_at, ds=export(db))
    mutate_game_result(db, game.id, home_score=200, away_score=0, attendance=99999)
    mutate_team_stats(db, game.id, pts=200)
    f2 = build_features(game.id, as_of=game.tipoff_at, ds=export(db))
    assert keys(f1) == keys(f2)
    assert all(approx_equal(f1[k], f2[k]) for k in f1)

def test_future_game_mutation_does_not_change_features(db, game):
    """as_of以降の試合を攪乱しても変わらないこと。"""
    f1 = build_features(game.id, as_of=game.tipoff_at, ds=export(db))
    mutate_games_after(db, game.tipoff_at)
    f2 = build_features(game.id, as_of=game.tipoff_at, ds=export(db))
    assert all(approx_equal(f1[k], f2[k]) for k in f1)

def test_as_of_is_actually_applied(db, game):
    """as_ofを前にずらすと、結果依存の特徴量が変化すること（陽性確認）。"""
    f_now = build_features(game.id, as_of=game.tipoff_at, ds=export(db))
    f_past = build_features(game.id, as_of=game.tipoff_at - timedelta(days=30),
                             ds=export(db))
    changed = [k for k in f_now if not approx_equal(f_now[k], f_past.get(k))]
    assert any(k.startswith(("elo_", "winrate_", "margin_")) for k in changed)

def test_backtest_uses_production_data_as_of(db):
    """保存済み feature_snapshot と再構築結果が一致すること。"""
    for p in sample_final_predictions(db, n=50):
        rebuilt = build_features(p.game_id, as_of=p.as_of,
                                  ds=export_at(db, p.data_as_of))
        assert approx_equal_dict(json.loads(p.feature_snapshot), rebuilt)

def test_features_exclude_unfinished_games(db, game): ...
def test_features_exclude_in_progress_game(db, game): ...

def test_accuracy_within_plausible_band(metrics):
    assert metrics.baseline_accuracy < metrics.accuracy < 0.85

def test_sql_spy_captures_bound_parameters():
    """sql_spy がバインド値も記録することの自己テスト。"""

```

旧版のテストは論理が逆だった。as_of を7日後にずらして「値が変わらないこと」を assert していたが、as_of を後ろにずらせば参照できる過去試合が増えるため、正しい実装なら値は変わる。あのテストは正しい実装を落とし、as_of を一切見ない実装だけを通していた。

ミューテーション試験: 意図的にリークさせた実装（elo_home に当該試合の得点差を混入）を用意し、リークテストが落ちることを CI で確認する（テストのテスト）。

sql_spy はクエリ文字列だけでなくバインドパラメータも記録する。プレースホルダを使えば game_id はクエリ文字列に現れず、文字列一致では検出できない。

6.2 freeze の検証

```

def test_finalize_at_tipoff_boundary():          # tipoff-1秒 / ちょうど / +1秒
def test_internal_predictions_rejects_after_tipoff():  # 409、1行も書き込まれない
def test_internal_predictions_rejects_when_is_final():
def test_gameday_update_skips_started_games():
def test_reinference_appends_not_replaces():  # 2行になり旧行が is_active=0
def test_cron_delay_simulation():              # tipoff+5分に実行しても確定予測が不変
def test_update_final_prediction_raises():      # トリガによる拒否

```

6.3 冪等性

```

def test_daily_ingest_twice_produces_identical_db():
    """ingestion_logs を除く全テーブルの行数と内容ハッシュが一致すること。"""
def test_daily_ingest_resumes_after_injected_failure():
def test_evaluate_is_idempotent():              # 3回実行して1行のまま
def test_finalize_is_idempotent():              # 2回目が is_final=1 行を UPDATE しない
def test_upsert_game_preserves_id():           # 日程変更で id が変わらない
def test_recompute_ratings_full_rebuild():     # 遡及投入後の洗い替え
def test_series_game_no_stable_under_reingest():
def test_entries_are_replaced_not_merged():    # 推定行が残らない

```

6.4 境界値・異常系

```

def test_tov_rate_with_null_possessions():      # None を返し、関数内で0埋めしない
def test_oreb_rate_zero_denominator():
def test_efg_zero_fga():
def test_attendance_rate_null_capacity():
def test_postponed_game_does_not_duplicate():
def test_cancelled_game_excluded_from_accuracy():  # outcome='VOID'
def test_rest_days_clipped_across_season_boundary():
def test_winrate_l5_does_not_cross_season_boundary():
def test_elo_season_regression_applied_once():    # 洗い替え2回で二重適用しない
def test_new_club_shows_early_season_badge():     # team_ratings に行がなくても
def test_all_starters_out_probability_clamped():  # [0.05, 0.95] にクランプ
def test_player_avail_below_threshold_excluded():  # availProb < 0.5 は非表示
def test_win_probs_sum_to_one_after_calibration():
def test_displayed_percentages_sum_to_100():      # 丸め後
def test_win_prob_and_score_agree():              # 符号の一致
def test_parse_minutes_mms() / test_parse_dnp() / test_parse_fullwidth_digits()
def test_parse_box_score_identities()             # pts = 2fgm×2 + 3fgm×3 + ftn
def test_null_score_game_raises_not_silently_zero()

```

6.4.1 個人スタッツの整合性（必須）

```

def test_team_targets_are_feasible(game):
    """チーム目標が制約と恒等式を満たすこと。整合化の前提条件。"""
    for t in team_targets(game):
        assert t.fg2a >= 0 and t.fg3a >= 0 and t.fta >= 0
        assert 0 <= t.fg2_pct <= 1 and 0 <= t.fg3_pct <= 1 and 0 <= t.ft_pct <= 1
        pts = t.fg2_pct * t.fg2a * 2 + t.fg3_pct * t.fg3a * 3 + t.ft_pct * t.fta
        assert abs(pts - team_pred(game)[t.side]) < 0.5

def test_player_predictions_reconcile_to_team(game):
    """選手予測の合計がチーム予測と一致すること。"""
    ps = player_predictions(game)
    assert abs(sum(p.pts for p in ps) - team_pred(game).pts) < 0.5

```

```

assert abs(sum(p.minutes for p in ps) - 200.0) < 0.5
for s in ('fg2a', 'fg3a', 'fta', 'oreb', 'dreb', 'ast', 'tov',
          'stl', 'blk', 'pf', 'fd'):
    assert rel_error(sum(getattr(p, s) for p in ps), team_target(game)[s]) < 0.02
# 成功数はチーム目標の成功数に一致する (ロジットシフトの帰結)
for pct, att in (('fg2_pct', 'fg2a'), ('fg3_pct', 'fg3a'), ('ft_pct', 'fta')):
    made = sum(getattr(p, pct) * getattr(p, att) for p in ps)
    assert rel_error(made, team_target(game)[pct] * team_target(game)[att]) < 0.02

def test_player_prediction_identities(game):
    """ 恒等式と大小関係が予測値でも成立すること。 """
    for p in player_predictions(game):
        assert approx(p.pts, p.fg2m * 2 + p.fg3m * 3 + p.ftm)
        assert approx(p.fgm, p.fg2m + p.fg3m)
        assert approx(p.fga, p.fg2a + p.fg3a)
        assert approx(p.reb, p.oreb + p.dreb)
        # 成功数は率×試投数の導出値なので、構造的に必ず成立する
        assert p.fg2m <= p.fg2a and p.fg3m <= p.fg3a and p.ftm <= p.fta
        assert all(0 <= getattr(p, k) <= 1
                   for k in ('fg2_pct', 'fg3_pct', 'ft_pct'))
        assert all(getattr(p, s) >= 0 for s in COUNT_STATS)

@pytest.mark.parametrize("seed", range(1500))
def test_reconciliation_converges(seed):
    """ 1,500 ケースのランダム入力で3回反復後の誤差が許容内に収まること。

    実測の基準値: 項目誤差の99%点 0.27%、最大 1.87%、得点誤差 0.00000。
    """
    players, target = random_case(seed)
    m, x, pct, pts = reconcile(players, target, iters=3)
    assert abs(pts.sum() - target_points(target)) < 0.5
    for s in ATTEMPTS + COUNTS:
        assert rel_error(x[s].sum(), target[s]) < 0.02

def test_reconciliation_no_clipping():
    """ 整合化後の成功率が [0, 1] を出ないこと。クリップに頼っていないことの確認。 """

def test_reconciliation_raises_on_infeasible_target():
    """ 実行不能な目標に対して無言でクリップせず InfeasibleTargetError を投げること。 """

def test_no_plus_minus_prediction_column():
    """ +/- の予測列・API フィールドが存在しないこと。 """

def test_percentage_hidden_below_attempt_threshold():
    """ 試投数が閾値未満のとき pct が null で返ること。 """

def test_count_rates_learned_per_minute():
    """ カウント11項目の目的変数が per-minute であること。 """

def test_pct_models_not_per_minute():
    """ 成功率3項目が per-minute で学習されていないこと (率は時間当たりの量ではない)。 """

```

```
def test_overtime_minutes_not_assumed():
    """ 予測時点で延長を仮定せず、総出場時間が 200 分であること。 """
```

凍結の網羅（子テーブルにも例外を設けない）

```
def test_player_predictions_frozen_when_parent_final():
    """ 親が is_final=1 のとき player_predictions の UPDATE/DELETE が拒否されること。 """

def test_prediction_reasons_frozen_when_parent_final():
    """ 同上。根拠が後から差し替えられるなら不変性の主張が成立しない。 """

def test_team_targets_frozen_when_parent_final():
def test_model_bundle_frozen_when_parent_final():

def test_freeze_transition_is_allowed():
    """ is_final 0→1 の遷移自体は通ること（トリガが freeze を妨げないこと）。 """
```

スナップショットと無料枠

```
def test_snapshot_matches_d1(db):
    """ スナップショットの行数と主キー集合が D1 と一致すること。 """

def test_snapshot_manifest_hashes():
    """ MANIFEST.json の SHA256 が実ファイルと一致すること。 """

def test_training_reads_no_d1(monkeypatch):
    """ 学習パスで D1 クライアントが一度も呼ばれないこと。 """

def test_batch_size_within_query_limit():
    """ 各テーブルの max_rows_per_request が floor(100/列数)×40 以下で、
    1リクエストあたりの文数が 50 以下に収まること。 """

def test_artifact_size_guard():
    """ 1.5MB を超える artifact_text の登録が拒否されること。 """

def test_static_export_file_count():
    """ out/ のファイル数が 18,000 以下であること。 """
```

極端な欠場（minutes_lost が学習域外）に対しては、モデルではなく後処理で確率を [0.05, 0.95] にクランプする。外挿域では較正も効かず、「ホーム 100% – アウェイ 0%」と表示して外すと信頼が一度で失われる。

6.5 回帰

```
def test_feature_list_diff_is_reported():      # 新旧の差分と欠損率をログ出力
def test_golden_predictions_within_tolerance():# 代表100試合、|p_new - p_old| > 0.15 が10件超で失敗
def test_brier_compared_on_identical_window():
def test_calibration_error_definition():      # ECE 等頻度10ビンの定義を固定
def test_only_one_active_model_per_type():    # uq_model_active
def test_rollback_restores_previous_model_version():
def test_deterministic_training():           # 同一入力・同一seedで予測が完全一致
```

6.6 パーサ

`tests/fixtures/` には**構造のみを模し、値をダミーに置換した合成HTML**を置く。実サイトの文言・記事・画像参照を残さない（要件4.5.2）。CIで実サイト由来文字列の混入を検査する。

fixtureはスナップショットであるため、定義上サイト構造の変更を検知できない。検知は `parser-canary`（日次、実サイトの日程ページ1枚）が担う。

6.7 API・画面

対象	内容
API	Zod 契約、Bearer なしで401、tipoff 経過後に409、キャッシュヘッダ、入力検証で400
トークン	コントラスト比 4.5:1 以上の 単体テスト （目視では見落とす）
E2E	<code>test_theme_persists_across_reload</code> 、 <code>test_no_horizontal_scroll_at_375px</code> 、 <code>test_disclaimer_reachable_from_every_page</code>

画面スナップショットは1画面×2テーマに絞る。3画面×2テーマは、トークンを1つ変えるたびに6枚落ちる維持コストが個人開発に見合わない。

6.8 テストデータ

`db/seeds/test/` に架空8チーム×2シーズン（約300試合）の決定論的シードを置く。実在選手名を固定しない。pytestのスキーマは `db/migrations/*.sql` をそのまま in-memory SQLite に適用する（二重管理するとテストが本番を反映しなくなる）。`test_migrations_apply_cleanly` でマイグレーション自体も検証する。

7. セキュリティ詳細

7.1 public リポジトリでの禁止事項

- Cloudflare API トークン、`INGEST_TOKEN` / `FINALIZE_TOKEN` の値
- `.env` の実ファイル（`.env.example` のみ可）
- 取得した生HTML（要件4.5.2）
- 学習済みモデルバイナリ（D1 に格納する）

`.gitignore`：

```
.env
.env.local
.dev.vars
*.db
*.sqlite
batch/cache/
data/raw/
**/fixtures/**/*.*.html
*.html.gz
models/*.pkl
```

`D1_DATABASE_ID` は機密ではない（操作にはAPIトークンが別途必要）ため `wrangler.toml` に直書きしてよい。`d1_databases` の `database_id` は必須フィールドで環境変数注入が効かないため、旧版の「直書きしない」という制約は実現不能だった。

7.2 認証

```
const constantTimeEq = async (a: string, b: string) => {
  const enc = new TextEncoder();
  const [ha, hb] = await Promise.all([
    crypto.subtle.digest("SHA-256", enc.encode(a)),
    crypto.subtle.digest("SHA-256", enc.encode(b)),
  ]);
  return crypto.subtle.timingSafeEqual(ha, hb); // 長さが揃うのでハッシュ経由
};

const auth = (secretKeys: string[]) => async (c, next) => {
  const m = /^Bearer\s+(\S+)$/i.exec(c.req.header("Authorization")) ?? "";
  if (!m) return c.json({ error: { code: "UNAUTHORIZED", message: "..."} }, 401);
  for (const k of secretKeys) {
    if (c.env[k] && await constantTimeEq(m[1], c.env[k])) return next();
  }
  return c.json({ error: { code: "UNAUTHORIZED", message: "..."} }, 401);
};

app.use("/internal/predictions", auth(["INGEST_TOKEN", "INGEST_TOKEN_NEXT"]));
app.use("/internal/finalize", auth(["FINALIZE_TOKEN"]));
```

`.replace("Bearer ", "")` は Bearer がなくてもヘッダ全体を返すため、スキーム検証にならない。

7.3 漏洩経路の遮断

経路	対策
Actions のログ (public で全世界から閲覧可)	例外オブジェクトをそのまま出力しない
<code>ingestion_logs.error_message</code>	型名と自前メッセージのみ。URLはホストとパスのみでマスク
<code>/api/v1/health</code>	<code>error_message</code> を返さない
依存パッケージ	<code>permissions: contents: read</code> 、Dependabot、 <code>==</code> 固定
push	Secret scanning + Push protection (push 時点で止まる)

8. 運用手順

8.1 初期構築

```

# 1. D1 作成 (ロケーションヒントを apac に)
wrangler d1 create bpredict --location apac

# 2. マイグレーション
wrangler d1 migrations apply bpredict --local # ローカル検証
wrangler d1 migrations apply bpredict --remote

# 3. API デプロイ (backfill より先。書き込み経路が /internal/* のため)
cd api && wrangler deploy

# 4. マスタ投入 (幕等。ingestion_logs に job='seed_master' で記録)
python -m batch.jobs.seed_master

# 5. 過去データ取り込み (1日1シーズン。再開可能)
python -m batch.jobs.backfill --season 2016-17-B1

# 6. 全期間のレーティング洗い替え
python -m batch.jobs.recompute_ratings --full

# 7. 初回学習
python -m batch.jobs.train --initial

# 8. Web デプロイ (Pages は Git 連携で自動)

```

実装順序の都合で、API デプロイが backfill より先に来る。書き込み経路を `/internal/*` に一本化したため、API がないと backfill が実行できない。

8.2 ローカル開発

```

# ローカル D1 にテストシードを投入
wrangler d1 execute bpredict --local --file=db/seeds/dev/seed.sql

# API をローカル起動
cd api && npm run dev

# バッチをローカル API に向ける
API_BASE_URL=http://127.0.0.1:8787 INGEST_TOKEN=dev \
python -m batch.jobs.daily_ingest --dry-run

```

本番 D1 に書きながら開発しない。is_final を含む本番データを壊すリスクを常時抱えることになる。

8.3 障害対応

症状	確認	対応
更新遅延バナーが出る	<code>/api/v1/health</code>	Actions のログ
<code>parser-canary</code> が fail	対象URLのHTML	合成 fixture を作りテストを書いてから修正
パースエラー多発	<code>ingestion_logs.error_type</code>	同上
予測が出ない	<code>model_versions.is_active</code>	有効モデルの存在確認
精度が急落	<code>/accuracy</code> の推移、 <code>degraded</code> フラグ	直前のモデル変更を確認。ロールバック
Workers 429	<code>/health</code> の quota	キャッシュTTL延長、WAFルール確認
中止試合でバッチ失敗	<code>prediction_results.outcome</code>	VOID として記録されているか確認

8.4 モデルのロールバック

```
UPDATE model_versions SET is_active = 0
WHERE model_type = 'WINNER' AND target = '' AND league = 'PREMIER';
UPDATE model_versions SET is_active = 1 WHERE version = 'winner-v1.0.0';
```

uq_model_active により、この順序でのみ成功する。確定済み予測は変更しない。

9. 実装順序

順	作業	完了条件
0	Phase 0 の検証 （P0-1～P0-10）と未決事項 U-01 / U-05 / U-06 の確定	取得可否と名称が決まる
0b	実機検証 （P0-12 LightGBM サイズ / P0-13 batch() クエリ計上 / P0-14 Next.js ビルド / P0-15 Zod コスト）	設計の前提が数値で裏付けられる。外れた項目は設計を修正してから先へ進む
1	D1 スキーマとマイグレーション（トリガ・CHECK を含む）	migrations apply が成功、test_migrations_apply_cleanly が通る
2	マスタ整備（seed_master、club_source_ids の対応表）	旧B1と新リーグのクラブが紐付く
3	CI の構築（ci.yml、parser-canary.yml、ESLint Flat Config）	push でテストが走る。eslint . が動く
4	Workers API の骨格（/internal/* と認証・ガード・バッチサイズ上限）	Bearer なしで401、tipoff 経過後に409、test_batch_size_within_query_limit が通る
5	スクレイパとパーサ（値域検証を含む）	合成 fixture でテストが通る
6	backfill による過去データ取り込み + スナップショット書き出し	全シーズンが DB に入り、test_snapshot_matches_d1 が通る
7	特微量生成とリーク検証テスト（入力スナップショット）	DB攪乱法のテストが通る。ミューテーション試験も通る。test_training_reads_no_d1 が通る
8	勝敗モデルの学習と評価（ 経路A・Bの両方 ）	Elo単体ロジスティック回帰を Brier で上回る。P0-11 で採用経路を決定
9	推論と predictions 登録、静的JSON書き出し	予測が JSON に出る
10	公開API（動的クエリ）	/games?date= /accuracy が応答する
11	画面（今日の予測・試合詳細・結果）	予測と結果が表示される。out/ のファイル数が18,000以下
12	Margin/Total モデル	予想スコアが出る。勝率と矛盾しない
12b	TeamRate モデル （整合化の目標値）	test_team_targets_are_feasible が通る
12c	個人モデル4段（Avail → Minutes → Rates → 整合化）	フルボックススコアが出る。test_reconciliation_converges と test_player_predictions_reconcile_to_team が通る
13	SHAP のグループ集約	根拠が4グループで表示される
14	的中率ページ（校正の言い換えを含む）	accuracy_summary から表示される
15	テーマ・PWA・免責・プライバシーポリシー	受け入れ基準 A-01～A-18 をすべて満たす

工程0で取得できないと判明した項目は、該当する特徴量を無効化して先に進む。工程を止めない。

工程 12b を 12c より先に置く。 整合化はチーム側の目標値がなければテストすらできず、目標値が制約を満たしていなければ原理的に成立しない。個人モデルから作ると、整合化の実装時に「目標が実行不能」という問題を個人モデル側の欠陥と誤認する。

旧版では工程3（取得可否の検証）がスキーマ確定より後にあり、取得できない項目が判明した時点で確定済み DDL を直すことになっていた。また backfill が API 構築より前にあり、書き込み経路の一本化と矛盾していた。